# High-Performance Computing

## Flynn’s classification &
## Parallel performance models

**Martin Schreiber**
Université Grenobles Alpes

**part_4_90_flynn_and_parallel_performance_models.odp**

Université Grenoble Alpes

LABORATOIRE JEAN KUNTZMANN
MATHÉMATIQUES APPLIQUÉES - INFORMATIQUE

# Outline

- **Flynn's classification**

- Parallel performance models
  - Introduction
  - Speedup and efficiency
  - Strong and weak scaling
  - Amdahl's law
  - Gustafson's law

- Summary

# Flynn's taxonomy...

… is a characterization into 4 groups of parallel processing computers

| | | Data stream | |
|---|---|---|---|
| | | **S**ingle **D**ata | **M**ultiple **D**ata |
| **Instruction stream** | **S**ingle **I**nstruction | **SISD**<br>E.g. processors from the first generations | **SIMD**<br>E.g. Nowadays on every computer |
| | **M**ultiple **I**nstructions | **MISD**<br>E.g. Fault tolerant processors (e.g. in Satellites, Automotives, etc.) | **MIMD**<br>E.g. Multi-core superscalar processors, large-scale super computers |

# That’s it about Flynn

- Simply remember this classification
- It’s helpful to discuss parallelization approaches with others

# Outline

- Flynn's classification
- **Parallel performance models**
  - **Introduction**
  - Speedup and efficiency
  - Strong and weak scaling
  - Amdahl's law
  - Gustafson's law
- Summary

# Warmup

- Parallelization of uppermost importance nowadays!
- Important to gain **a-priori understanding** of **parallel performance of programs / algorithms**
- Parallel performance can be **modeled** with performance models
- The most famous ones are
  - **Amdahl's** law
  - **Gustafson's** law

# Outline

- Flynn's classification
- **Parallel performance models**
  - Introduction
  - **Speedup and efficiency**
  - Strong and weak scaling
  - Amdahl's law
  - Gustafson's law
- Summary

# Parallel speedup & efficiency

- Given performance results:
  - $T_1$ : Time for execution of program using one processor
  - $T_P$: Time for execution of program using “P” processors

- **Parallel speedup:**
  - A **speedup “S”** refers to factor of the wallclock time reduction by parallelization if comparing it to a single-processor execution.

$$S_P = \frac{T_1}{T_P}$$

  - Dimensionless number
  - Theoretical limit: For “P” processors, we get $S \in [1; P]$
    - Note: This is just a model and not always true in practical studies
    - E.g. super-linear speedup exists, why?

# Parallel speedup & efficiency

- Given performance results:
  - $T_1$ : Time for execution of program using one processor
  - $T_P$: Time for execution of program using “P” processors

- **Parallel efficiency:**
  - How efficient are the resources in parallel (0% - 100%)

$$E_P = \frac{T_P}{P}$$

  - Also a dimensionless number
  - Theoretical limit: $E_P \in [0; 1]$
    - Also here: This is just a model and not always true in practical studies
    - => Can exceed “1” for real systems

# Outline

- Flynn's classification
- **Parallel performance models**
  - Introduction
  - Speedup and efficiency
  - **Strong and weak scaling**
  - Amdahl's law
  - Gustafson's law
- Summary

# Strong and weak scaling

- **“Scaling”**: Execution and performance evaluation of a program on an increasing number of processors
- **Workload size can vary** during scaling:

**“Strong scaling”**

- The **problem size remains constant** throughout the scaling => **Total workload size remains constant**
- Example: Computer game with fixed resolution needs to run on more processors

**“Weak scaling”**

- The **problem size grows linearly with the number of processors** => **Per-processor workload size remains constant**
- Example: Increasing the resolution of a computer game relative to the number of used processors

# Strong and weak scaling

There are two main/famous parallel performance models:

- **Amdahl's law**:
  - Probably the first performance modeling law
  - Constant problem size (related to strong scaling)

- **Gustafson's law**:
  - Motivated by issues with Amdahl's law
  - Problem size grows with number of processors (related to weak scaling)

More about this on the next slide

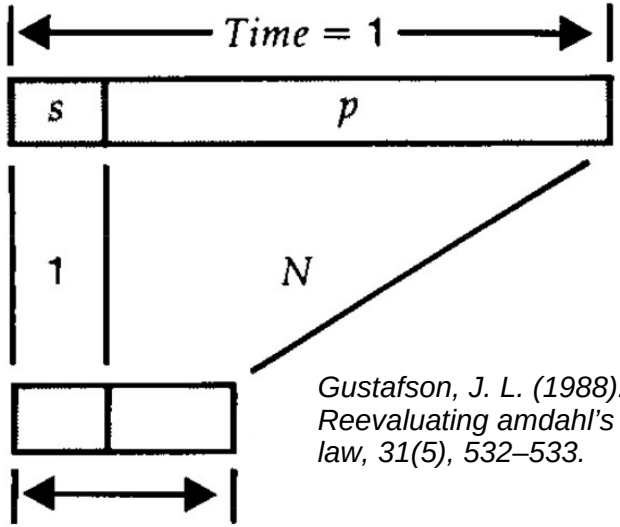
# Outline

- Flynn's classification
- **Parallel performance models**
  - Introduction
  - Speedup and efficiency
  - Strong and weak scaling
  - **Amdahl's law**
  - Gustafson's law
- **Summary**

# Amdahl's law

- Basic idea: Parallelization of programs with constant problem size (e.g., number of degrees of freedom)
- **Runtime** $T_{P=1} = t_p + t_s$ of program code is split into
  - sequential part $t_s$ and
  - parallelizable part $t_p$
- Often specified in normalized way $s + p = 1$
  - sequential part $s = t_s/(t_s + t_p)$ and
  - parallelizable part $p = t_p/(t_s + t_p)$
- **Runtime** of parallelizable part is distributed to "n" processors:

$$T_{P=n} = t_s + \frac{t_p}{n} \qquad \overline{T}_{P=n} = s + \frac{p}{n}$$




*Gustafson, J. L. (1988). Reevaluating amdahl's law, 31(5), 532–533.*

# Amdahl's law

- Speedup for constant problem size, based on
  - $T_{P=1}$: Runtime to execute a program on one processor
  - $T_{P=n}$: Runtime to execute a program on "n" processors

$$S(n) = \frac{T_{P=1}}{T_{P=n}} \qquad S(n) = \frac{\overline{T}_{P=1}}{\overline{T}_{P=n}} = \frac{1}{\overline{T}_{P=n}}$$

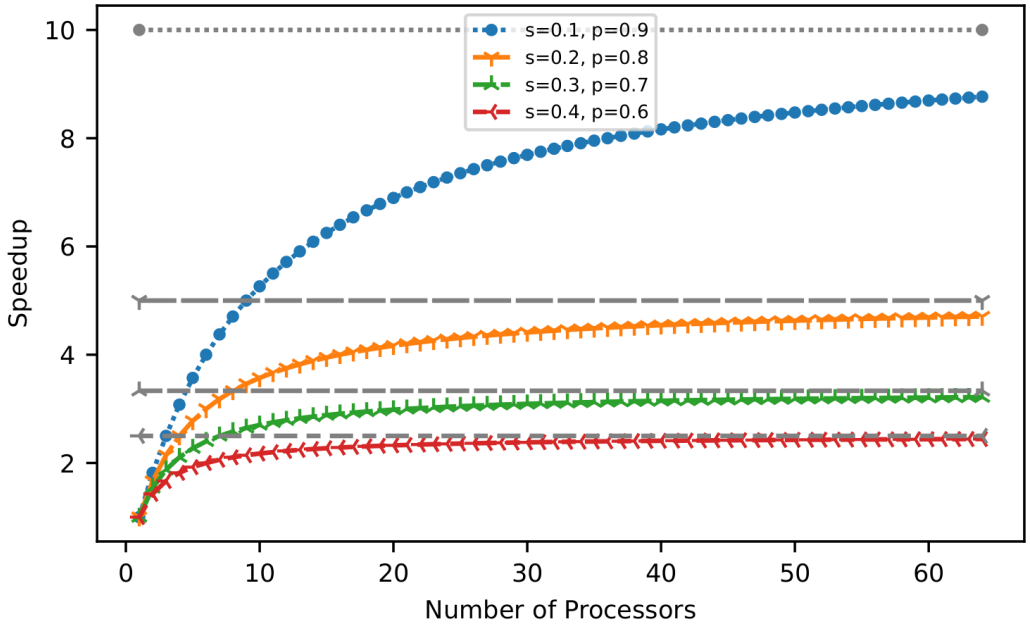
- **Amdahl's law** is then

$$S_{\mathrm{Amdahl}}(n) = \frac{t_p + t_s}{t_s + \frac{t_p}{n}} \qquad S_{\mathrm{Amdahl}}(n) = \frac{s + p}{s + \frac{p}{n}} = \frac{1}{s + \frac{p}{n}}$$

- Related to "Strong scaling":
  Problem where parallelization is applied to a fixed problem size

# Amdahl's law

- Speedup plots for
$$S(n) = \frac{1}{s + \frac{p}{n}}$$
given in the right image
- Scalability is asymptotically limited by
$$\lim_{n\to\infty} S_n = \lim_{n\to\infty} \frac{1}{s + \frac{p}{n}} = \frac{1}{s}$$
- This is an **upper bound** for strong scaling problems!
- **=> Adding more processors doesn't help much more!**

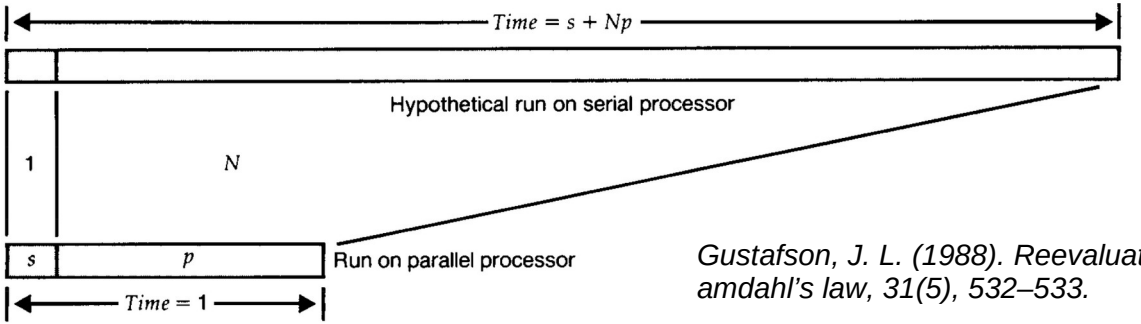


# Outline

- Flynn's classification
- **Parallel performance models**
  - Introduction
  - Speedup and efficiency
  - Strong and weak scaling
  - Amdahl's law
  - **Gustafson's law**
- Summary

# Gustafson's law

- Basic idea: **Problem size grows with number of processors**
- Runtime of program on one processor is given by
  - sequential part $t_s$
  - parallelizable part $t_p$ (on one processor!)




*Gustafson, J. L. (1988). Reevaluating amdahl's law, 31(5), 532–533.*

Examples for practical applications:

- Realtime processing: Signal processing
  More processing power => more data can be processed
- Computer Games (Higher resolution => Higher requirements on parallel cores on GPU)

# Gustafson's law

**W: Factor for workload**
**P: Number of processors**

- Time on one processor (Attention: We don't use it here!)

$$T_{P=1} = t_s + t_p \qquad \overline{T}_{P=1} = s + p = 1$$

- Time/Workload „W“ with „n“-times the problem size:
  - on one processor

$$T_{P=1}^{W=n} = t_s + n \cdot t_p \qquad \overline{T}_{P=1}^{W=n} = s + n \cdot p$$

  - on „n“ processors

$$T_{P=n}^{W=n} = t_s + n \cdot t_p/n = t_s + t_p \qquad \overline{T}_{P=1}^{W=n} = s + n \cdot p/n = s + p = 1$$

- Speedup:

$$S_{\mathrm{Gutsafson}}(n) = \frac{T_{P=1}^{W=n}}{T_{P=n}^{W=n}} = \frac{t_s + n \cdot t_p}{t_s + t_p}$$

$$S_{\mathrm{Gutsafson}}(n) = \frac{\overline{T}_{P=1}^{W=n}}{\overline{T}_{P=n}^{W=n}} = \frac{s + n \cdot p}{s + p} = s + n \cdot p$$

# Gustafson's law

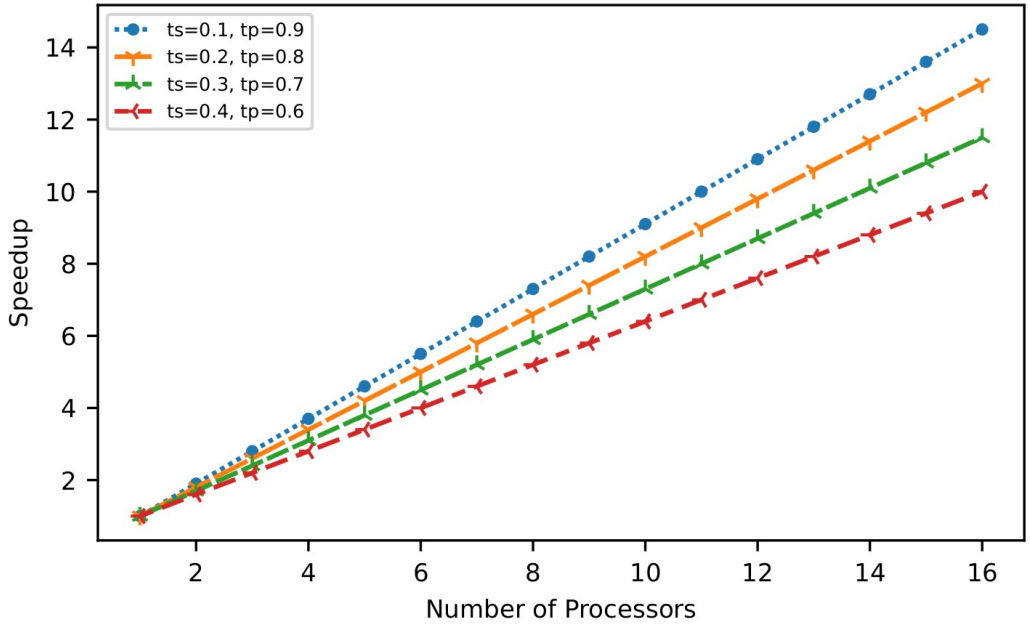
- Speedup plots for

$$S(n) = s + n \cdot p$$

given in the right image

- Scalability is **asymptotically** obviously **not limited**
- We observe a **linear scalability**

- **=> Adding more processors always leads to better scalability!**



# Outline

- Flynn’s classification
- Parallel performance models
  - Introduction
  - Speedup and efficiency
  - Strong and weak scaling
  - Amdahl’s law
  - Gustafson’s law
- **Summary**

# Summary

- Flynn’s taxonomy used to characterize different HPC systems
- Speedup & efficiency
- Performance models:
  - Gain (theoretical) understanding in parallel performance of developments
  - **Amdahl**: Fixed workload
  - **Gustafson**: Variable workload

# High-Performance Computing

Based on slides of Thomas Ropars + the book "Parallel Algorithms" (by Casanova, Robert, and Legrand)

## Performance modeling of distributed memory algorithms (Part I)

**Martin Schreiber**
Université Grenobles Alpes

**part_3_81_EXTRA_perf_models_distr_mem_part_1.odp**

Université Grenoble Alpes

Inria

LABORATOIRE JEAN KUNTZMANN
MATHÉMATIQUES APPLIQUÉES - INFORMATIQUE

# Outline

- **Warmup**
- Examples
  - Dot product
  - Matrix-vector multiplication
  - Matrix-matrix multiplication
  - Matrix-matrix with matrix-vector
- Scalability comparison
- Discussion

# Warmup

- **Definition of “Processor”**
  - The **performance modeling** we’ll do here goes back **several decades**
  - At this time, there was **just one processor per compute node**
  - To stick to this same nomenclature as it has been used decades ago,
    - a **processor** refers to one **compute node** and
    - a processor is the **smallest computational entity** on the computer
- **Network:**
  - In the remainder of this presentation we assume a **ring network**
  - In all algorithms, the communication will be only done to the previous or next node, hence **just requiring one HOP**

# Outline

- Warmup
- **Examples**
  - **Dot product**
  - Matrix-vector multiplication
  - Matrix-matrix multiplication
  - Matrix-matrix with matrix-vector
- Scalability comparison
- Discussion

# Example: Dot product

- Challenge: Which **hardware to buy next** to run your algorithms?
- Warmup example: Let's take a **dot product**

$$c = \sum_{i=1}^{n} a_i b_i$$

- How can we determine the **time on a compute cluster which doesn't yet exist** to run this computation (without running it…)?

- Potential solution: Investigate the **parallel performance of algorithms** with **simplified parallel performance models**


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Performance model

- **Parameters** of **parallel algorithm**:
  - “**p**”: Number of processors
  - “**n**”: Vector size
  - “**r**” = n/p: Size of each piece of vector where “r” should be an integer number

- **Parameters** given by the **hardware**:
  - “**L**”: Latency of network communication
  - “**B**”: Bandwidth of network communication
  - “**A**”: Bandwidth of node-local network
  - “**w**”: The computation time for one arithmetic operation (mul or add)

- Communication: **Reduce operation** at the end of each iteration

# Example: Dot product

$$c = \sum_{i=1}^{N} a_i b_i$$

## Computational time:

- Counting the **arithmetic operations**, we could, e.g.,
  - use the **exact number of mathematical operations**:

$$F^*(n, p) = 2r - 1$$

  - use an **approximation** (Which would be better the larger “n” is)

$$F(n, p) = 2r$$

- For the **total time** for the computations we could
  - try to understand the full computer architecture or
  - **assume** that **each operation takes time “w”** to get

$$T_{\mathrm{comp}}(n, p) = w \cdot F(n, p) = 2wr$$


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Example: Dot product

$$c = \sum_{i=1}^{N} a_i b_i$$

**Data loading time (on-node/processor):**

- We also need to **load and store the data for each processor**
- We need to load **2 vectors of size "r"** ("A": Node-local bandwidth)

$$T_{\mathrm{data}}(n, p) = \frac{2r}{A}$$

**Data communication time (between nodes / processors):**

- Again, we can use a simplified model:
- "**r**" = N/p: Workload per processor
- Time for reduction (assuming tree-based reduction)

$$T_{\mathrm{comm}}(n, p) = \underbrace{\log(p)L}_{\text{tree network}} + \underbrace{B^{-1}}_{\text{One number}}$$


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Example: Dot product

$$c = \sum_{i=1}^{N} a_i b_i$$

## Total time for single processor (p=1):

- We assume that per-processor computations and data load/store happens **asynchronously**

$$T_{\text{processor}}(n, p) = \max\left(T_{\text{comp}}(n, p), T_{\text{data}}(n, p)\right)$$

- This is then **followed by the communication** (without any asynchronous operation possible):

$$\begin{aligned} T_{\text{total}}(n, p) &= T_{\text{processor}}(n, p) + T_{\text{comm}}(n, p) \\ &= \max\left(T_{\text{comp}}(n, p), T_{\text{data}}(n, p)\right) + T_{\text{comm}}(n, p) \end{aligned}$$


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Example: Dot product

- In total, we get the following performance model

$$
\begin{aligned}
T_{\text{total}}(n,p) &= \max\left(T_{\text{comp}}(n,p), T_{\text{data}}(n,p)\right) + T_{\text{comm}}(n,p) \\
&= \max\left(\frac{2wn}{p}, \frac{2n}{Ap}\right) + \log(p)L + B^{-1} \\
&\approx \frac{2n}{Ap} + \log(p)L
\end{aligned}
$$

$r = \frac{n}{p}$

- We applied two simplifications in the last step:
  - The first one is related to dropping the reciprocal of “B” – Why?
  - The second one is related to skipping the computational part – Why?


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Outline

- Warmup
- **Examples**
  - Dot product
  - **Matrix-vector multiplication**
  - Matrix-matrix multiplication
  - Matrix-matrix with matrix-vector
- Scalability comparison
- Discussion

# Example: Matrix-Vector multiplication

- Given:
  - Matrix “A” of size $n \times n$
  - Vectors “x” and “y” of size n
- We investigate

$$y = Ax$$

$$\begin{bmatrix} y_1 \\ y_2 \\ \vdots \\ y_n \end{bmatrix} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix} \times \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix}$$

- **Task: We should parallelize this algorithm with “P” processors**

- Pseudo code:

```
for(int i=0; i<n; i++) {
     y[i] = 0;
     for(int j=0; j<n; j++) {
          y[i] = y[i] + A[i*n+j] * x[j]
     }
}
```

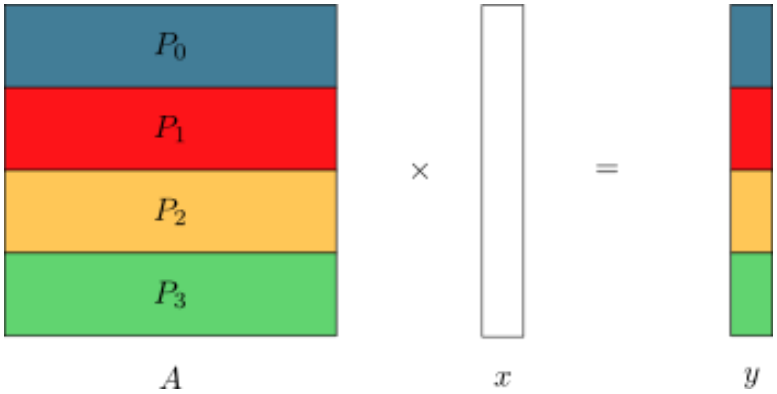
- Observation:
  - Each output element in “y” requires a dot product
  - Each output element is independent

- => Parallelize each row (Next slide)


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Example: Matrix-Vector multiplication

- Parallelization approach:
  - **Distribute rows** of Matrix “A”
  - **Distribute vector** “y”
  - **Duplicate** “x” to all processor
- Example with p=4 processors and each color referring to a different processor




- Result:
  - Data needs to be **distributed at the beginning**
  - Then, **we don’t require any communication at all**
  - **Rather “boring” example**
- Next slide: More challenging task


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

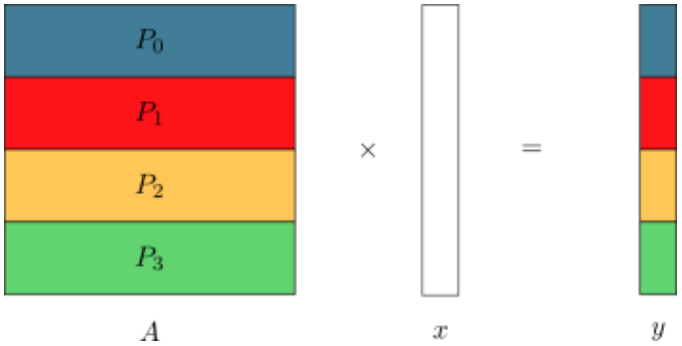
# Example: Matrix^N-Vector multiplication

- How about if we **repeatedly compute matrix-vector multiplications**?

$$x^{n+1} = Ax^n$$

$$x^3 = Ax^2 = A(Ax^1) = A(A(Ax^0)))$$

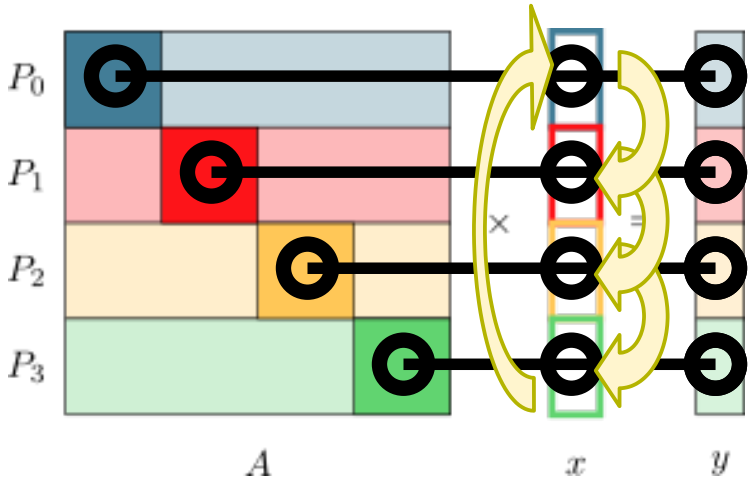
- Problem: Algorithm requires entire "x" on each processor




- Solution #1 (**naive** solution):
  - We can **broadcast** the " $x^{n+1}$ " vector after each iteration…
  - Depending on network **topology**, this can have significant overheads due to longer communication distances
- **Solution #2:**
  - Instead of broadcasting "x", we **forward** "our" part of "x" we know to the **next processor**
  - After "p" **iterations**, we're done

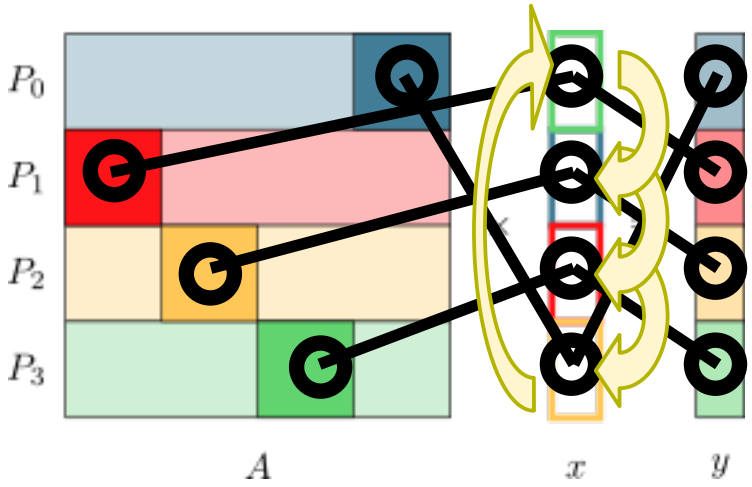
# Example: Matrix^N-Vector multiplication

- Each processor computes one part of the output vector “y”
- **Iteration 1**
  - a part of “x” is used to compute the partial matrix-vector multiplication
  - this part of “x” is send to the next processor



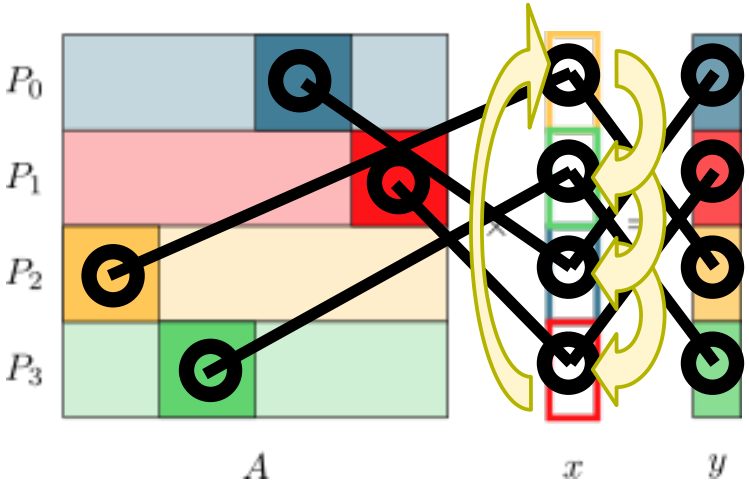
# Example: Matrix^N-Vector multiplication

- Each processor computes one part of the output vector “y”
- **Iteration 2**
  - a part of “x” is used to compute the partial matrix-vector multiplication
  - this part of “x” is send to the next processor



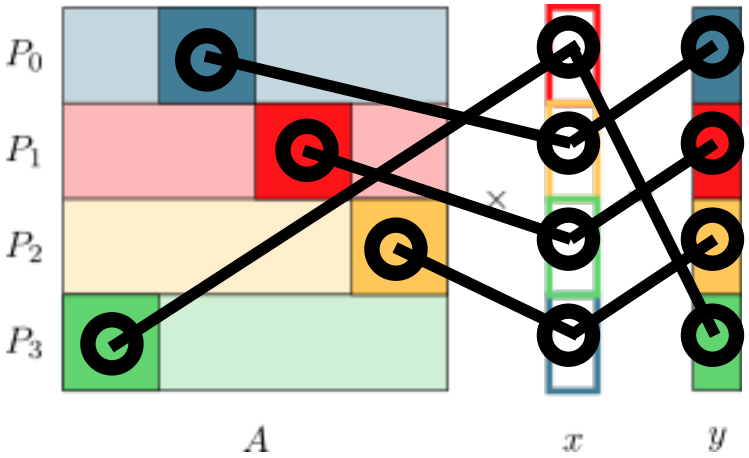
# Example: Matrix^N-Vector multiplication

- Each processor computes one part of the output vector “y”
- **Iteration 3**
  - a part of “x” is used to compute the partial matrix-vector multiplication
  - this part of “x” is send to the next processor



# Example: Matrix^N-Vector multiplication

- Each processor computes one part of the output vector “y”
- **Iteration 4**
  - a part of “x” is used to compute the partial matrix-vector multiplication
  - ~~this part of “x” is send to the next processor~~ (or maybe yes)



# Communication

- Obviously, this way of communication doesn't perform that well
  - We use **blocking send & recv**
  - We **send data at the end of the iteration**

- New attempt:
  - Use **non-blocking send & recv**
  - We send the data **as soon as possible**
    => at the **beginning of iteration**

- Non-blocking send/recv allows us to do computation and computation in parallel **(ALWAYS try to do this!)**


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Performance model

- **Parameters** of **parallel algorithm**:
  - “**p**”: Number of processors
  - “**n**”: Matrix size
  - “**r**” = n/p: Size of each matrix partition where “r” should be an integer number
- **Parameters** given by the **hardware**:
  - “**L**”: Latency of network communication
  - “**B**”: Bandwidth of network communication
  - “**w**”: The computation time for one arithmetic operation (mul or add)
- Communication:
  - Send and receive of same message takes same time
  - **Non-blocking communication** at the beginning of each iteration


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Performance model

- For **one iteration**:
  - Time for **computation**

$$T_{comp} = \underbrace{r^2}_{\text{One block}} \cdot \underbrace{2 \cdot w}_{\text{mul + add}}$$

  - Non-blocking communication of **chunks of "x"**

$$T_{comm} = L + \underbrace{r}_{\text{Chunk of vector}} \frac{1}{B}$$

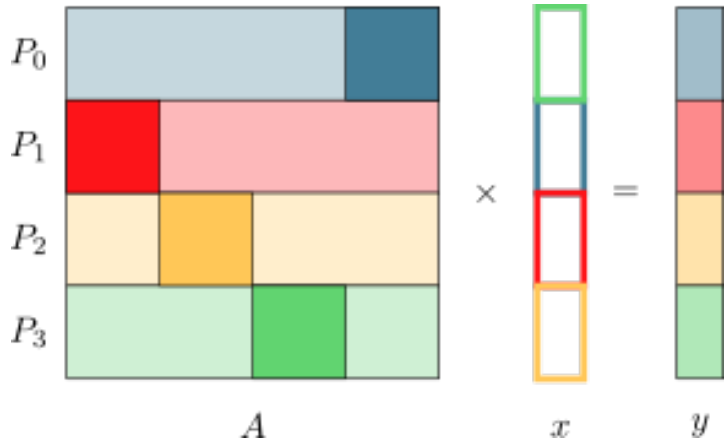
  - The **total wallclock** time for one iteration is then

$$T_{iter} = \max\left(T_{comp}, T_{comm}\right)$$

- Time for **p iterations** (full matrix-vector multiplication:

$$\begin{aligned} T_{MV}(p,n) &= p \cdot \max\left(T_{comp}, T_{comm}\right) \\ &= p \cdot \max\left(r^2 \cdot 2 \cdot w, L + \frac{r}{B}\right) \\ &= \max\left(\frac{2n^2 w}{p}, pL + \frac{n}{B}\right) \end{aligned}$$

$r = \frac{n}{p}$



# Performance model

For **very large matrices**:

- Investigate the performance model for very large matrices:

$$\lim_{n\to\infty} T_{MV}(n,p) = \lim_{n\to\infty} \max\left(\frac{2n^2w}{p}, pL + \frac{n}{B}\right)$$
$$= \lim_{n\to\infty} \frac{2n^2w}{p}$$

- Workload “perfectly” distributed
- **=> Optimal scaling**

For **large number of processors**:

- Investigate performance model for large number of processors:

$$\lim_{p\to\infty} T_{MV}(n,p) = \lim_{p\to\infty} \max\left(\frac{2n^2w}{p}, pL + \frac{n}{B}\right)$$
$$= \lim_{p\to\infty} pL + \frac{n}{B}$$

- The latency will increase linearly with the number of processors
- **=> Sub-optimal scaling dominated by latency**


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Outline

- Warmup
- **Examples**
  - Dot product
  - Matrix-vector multiplication
  - **Matrix-matrix multiplication**
  - Matrix-matrix with matrix-vector
- Scalability comparison
- Discussion

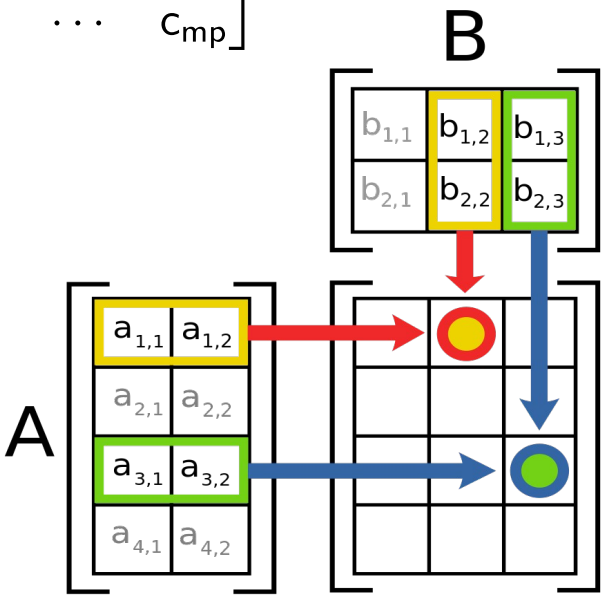
# Example: Matrix-matrix multiplication (mat-mat-mul)

- Next, we investigate the matrix-matrix multiplication given by $C = AB$

$$\begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \times \begin{bmatrix} b_{11} & b_{12} & \cdots & b_{1p} \\ b_{21} & b_{22} & \cdots & b_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ b_{n1} & b_{n2} & \cdots & b_{np} \end{bmatrix} = \begin{bmatrix} c_{11} & c_{12} & \cdots & c_{1p} \\ c_{21} & c_{22} & \cdots & c_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ c_{m1} & c_{m2} & \cdots & c_{mp} \end{bmatrix}$$

- In Pseudo code:

```
for (i=0; i < n ; i++) {
  for (j=0; j < n; j++) {
    C [i*n+j] = 0;
    for (k =0; k < n; k++) {
      C[i*n+j] = C[i*n+j] + A[i*n+k]*B[k*n+j];
    }
  }
}
```



# Example: Matrix-matrix multiplication (mat-mat-mul)

- Assume that after one mat-mat-mul we need to do other mat-mat-muls, e.g.,

$$D = EC = E(AB)$$

- We assume **block-wise matrix multiplications**

$$\left[\begin{array}{cc|ccc} c_{11} & c_{12} & \cdots & & c_{1p} \\ c_{21} & c_{22} & \cdots & & c_{2p} \\ \hline \vdots & \vdots & \ddots & & \vdots \\ c_{m1} & c_{m2} & \cdots & & c_{mp} \end{array}\right] = \left[\begin{array}{cc|ccc} a_{11} & a_{12} & \cdots & & a_{1n} \\ a_{21} & a_{22} & \cdots & & a_{2n} \\ \hline \vdots & \vdots & \ddots & & \vdots \\ a_{m1} & a_{m2} & \cdots & & a_{mn} \end{array}\right] \times \left[\begin{array}{cc|ccc} b_{11} & b_{12} & \cdots & & b_{1p} \\ b_{21} & b_{22} & \cdots & & b_{2p} \\ \hline \vdots & \vdots & \ddots & & \vdots \\ b_{n1} & b_{n2} & \cdots & & b_{np} \end{array}\right]$$

- For the 2nd row and 4 processors, the mat-mat-mul with blocks would be:

$$C_{2,1} = A_{2,1}B_{1,1} + A_{2,2}B_{2,1} + A_{2,3}B_{3,1} + A_{2,4}B_{4,1}$$
$$C_{2,2} = A_{2,1}B_{1,2} + A_{2,2}B_{2,2} + A_{2,3}B_{3,2} + A_{2,4}B_{4,2}$$
$$C_{2,3} = A_{2,1}B_{1,3} + A_{2,2}B_{2,3} + A_{2,3}B_{3,3} + A_{2,4}B_{4,3}$$
$$C_{2,4} = A_{2,1}B_{1,4} + A_{2,2}B_{2,4} + A_{2,3}B_{3,4} + A_{2,4}B_{4,4}$$

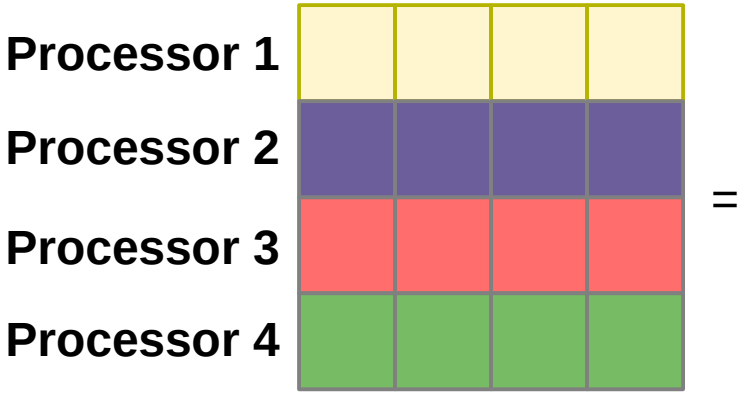

Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Example: Matrix-matrix multiplication

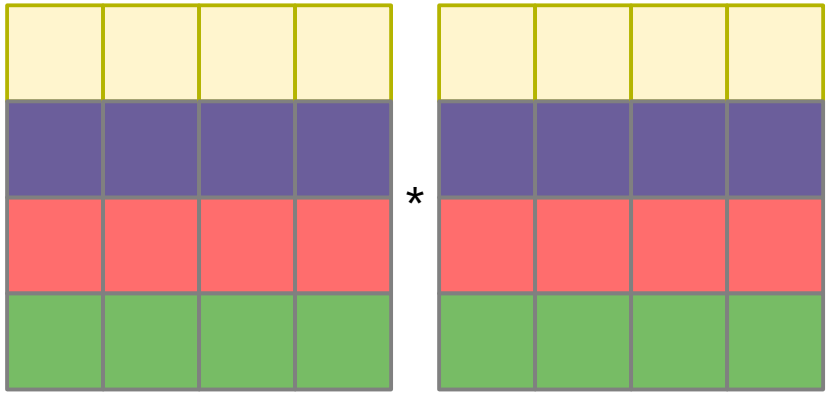
Algorithmic idea (similar to previous matrix-vector multiplication)

- **Distribute rows** of all matrices across processors, then each processor has

$$r = \frac{n}{p}$$




- For “p”-th **iteration**:
  - Compute **partial mat-mat-mul** based on available data
  - Each processor **sends its owned** row of “B” to the next one

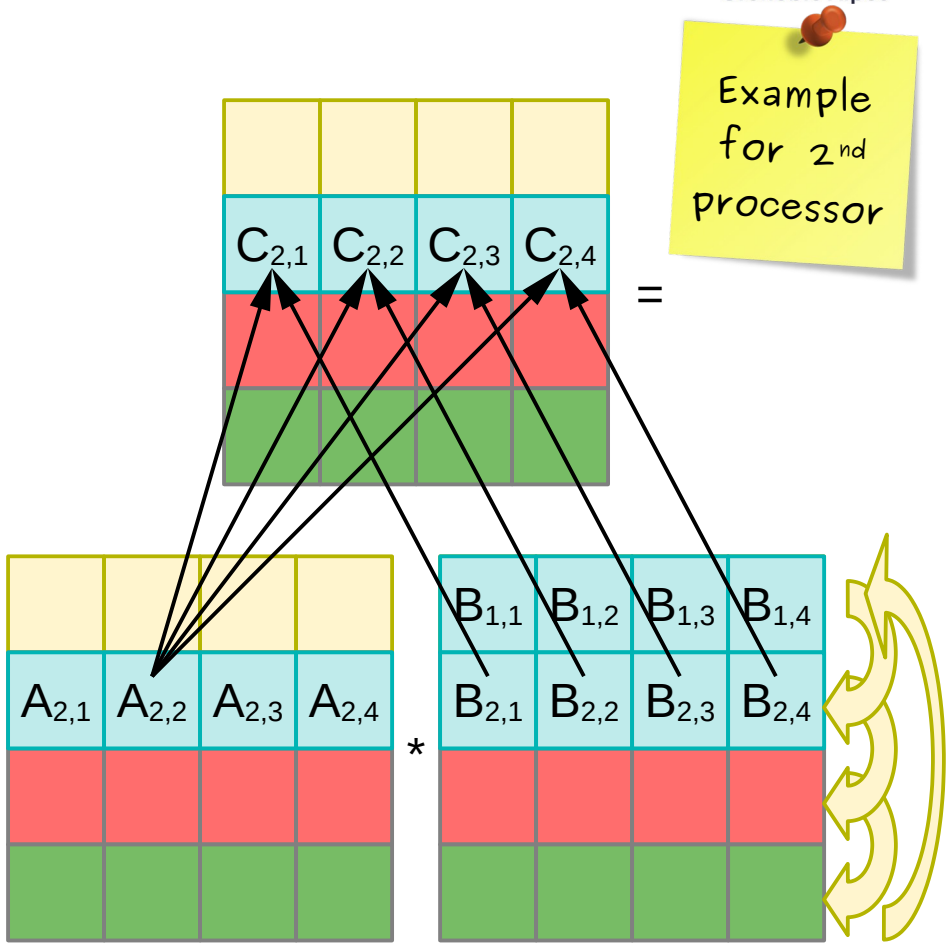


# Iteration 1

- Compute partial updates, e.g.,

$$C_{2,1} = A_{2,1}B_{1,1} + \cancel{A_{2,2}B_{2,1}} + A_{2,3}B_{3,1} + A_{2,4}B_{4,1}$$
$$C_{2,2} = A_{2,1}B_{1,2} + \cancel{A_{2,2}B_{2,2}} + A_{2,3}B_{3,2} + A_{2,4}B_{4,2}$$
$$C_{2,3} = A_{2,1}B_{1,3} + \cancel{A_{2,2}B_{2,3}} + A_{2,3}B_{3,3} + A_{2,4}B_{4,3}$$
$$C_{2,4} = A_{2,1}B_{1,4} + \cancel{A_{2,2}B_{2,4}} + A_{2,3}B_{3,4} + A_{2,4}B_{4,4}$$

- Send B-row to next processor

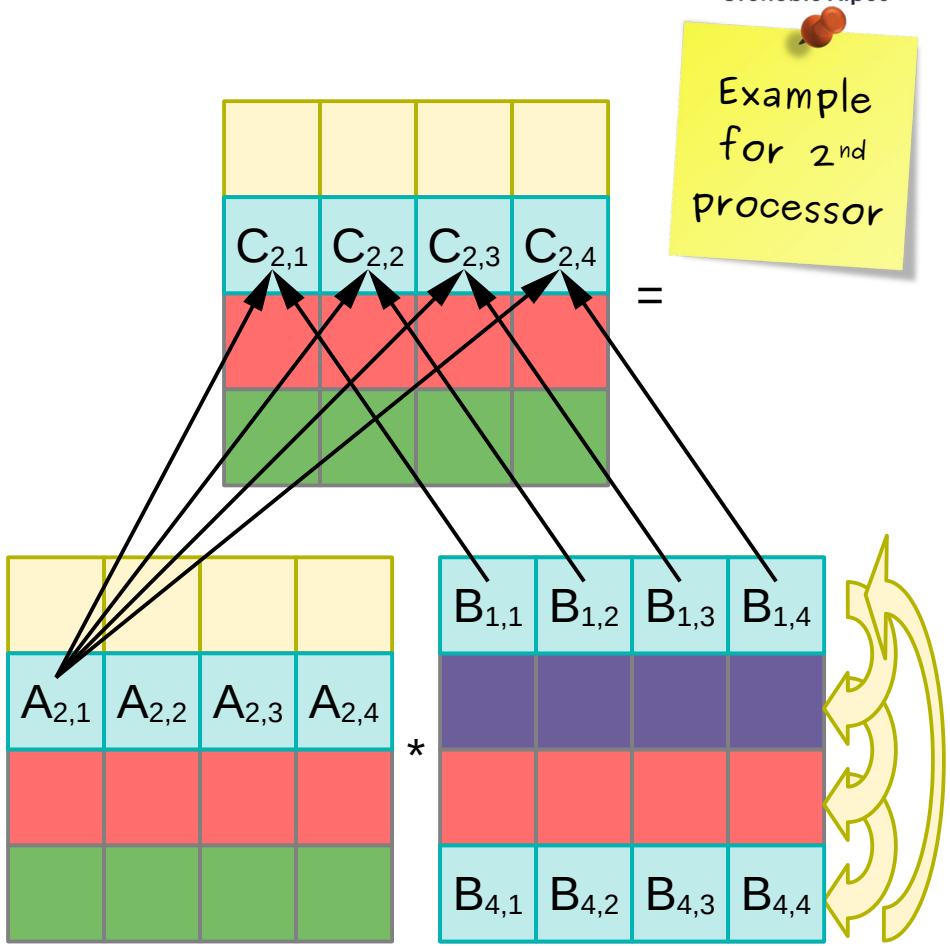


# Iteration 2

- Compute partial updates, e.g.,

$$C_{2,1} = \cancel{A_{2,1}B_{1,1}} + \cancel{A_{2,2}B_{2,1}} + A_{2,3}B_{3,1} + A_{2,4}B_{4,1}$$
$$C_{2,2} = \cancel{A_{2,1}B_{1,2}} + \cancel{A_{2,2}B_{2,2}} + A_{2,3}B_{3,2} + A_{2,4}B_{4,2}$$
$$C_{2,3} = \cancel{A_{2,1}B_{1,3}} + \cancel{A_{2,2}B_{2,3}} + A_{2,3}B_{3,3} + A_{2,4}B_{4,3}$$
$$C_{2,4} = \cancel{A_{2,1}B_{1,4}} + \cancel{A_{2,2}B_{2,4}} + A_{2,3}B_{3,4} + A_{2,4}B_{4,4}$$

- Send B-row to next processor

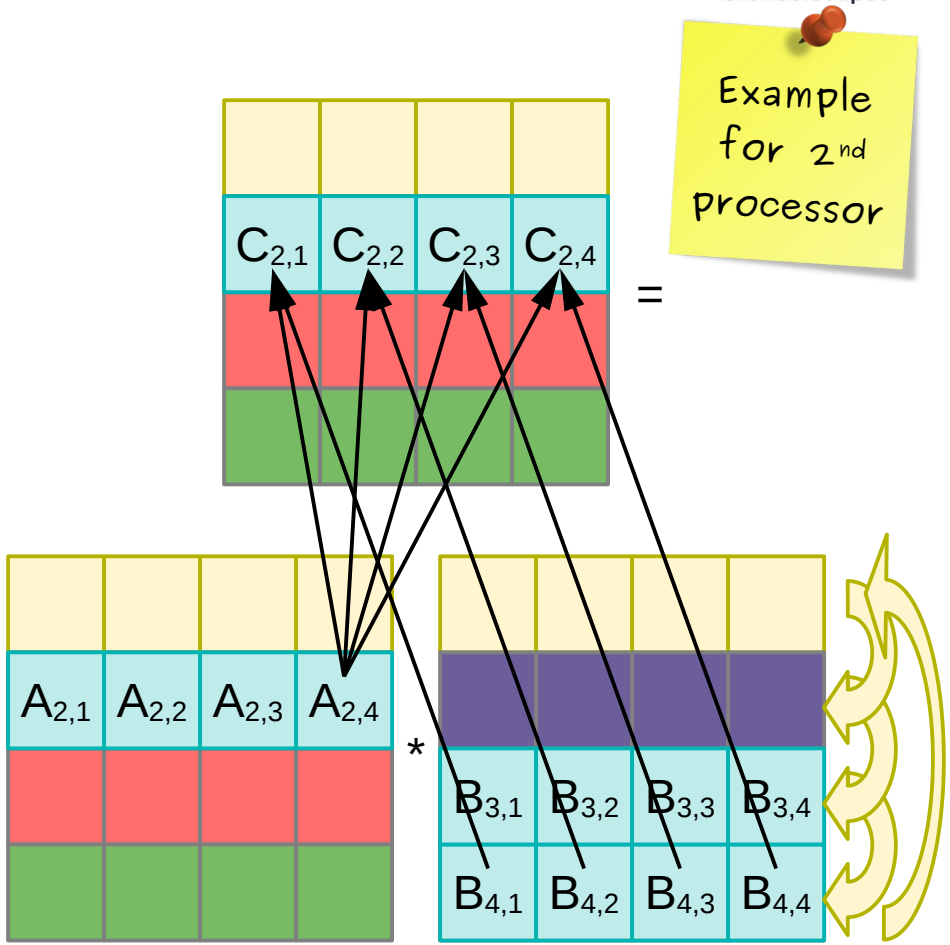


# Iteration 3

- Compute partial updates, e.g.,

$$
\begin{aligned}
C_{2,1} &= \cancel{A_{2,1}B_{1,1}} + \cancel{A_{2,2}B_{2,1}} + A_{2,3}B_{3,1} + \cancel{A_{2,4}B_{4,1}} \\
C_{2,2} &= \cancel{A_{2,1}B_{1,2}} + \cancel{A_{2,2}B_{2,2}} + A_{2,3}B_{3,2} + \cancel{A_{2,4}B_{4,2}} \\
C_{2,3} &= \cancel{A_{2,1}B_{1,3}} + \cancel{A_{2,2}B_{2,3}} + A_{2,3}B_{3,3} + \cancel{A_{2,4}B_{4,3}} \\
C_{2,4} &= \cancel{A_{2,1}B_{1,4}} + \cancel{A_{2,2}B_{2,4}} + A_{2,3}B_{3,4} + \cancel{A_{2,4}B_{4,4}}
\end{aligned}
$$

- Send B-row to next processor

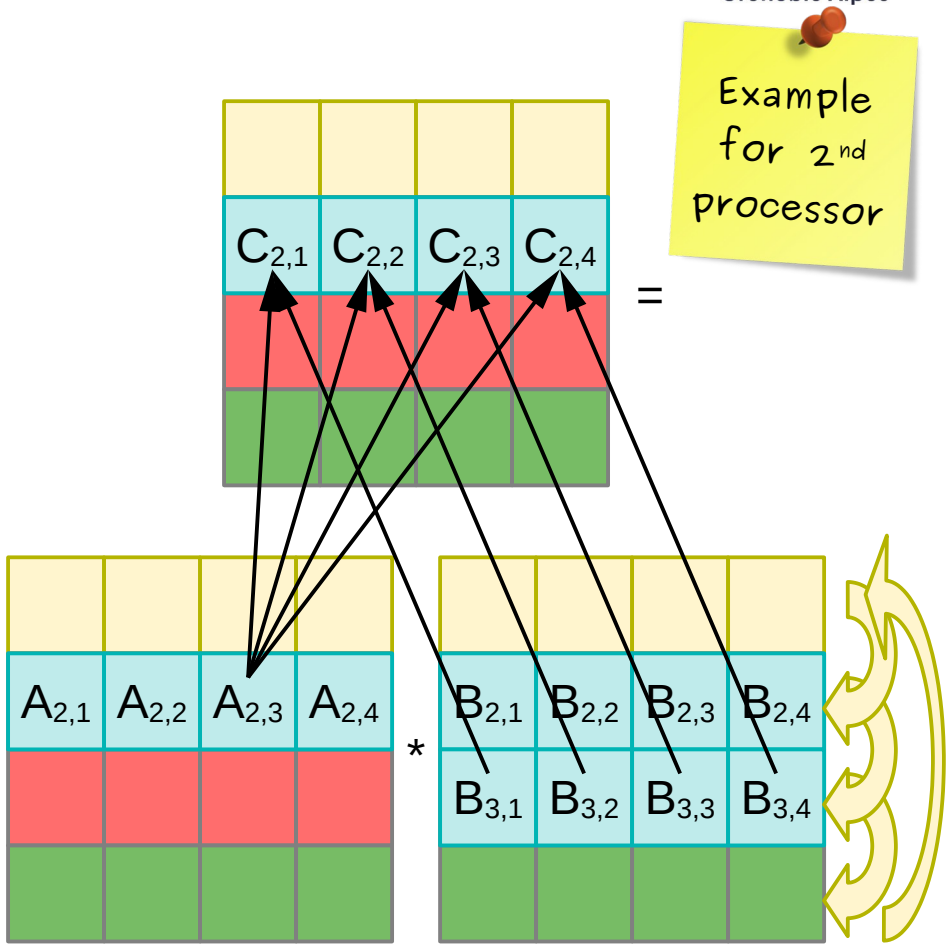


# Iteration 4

- Compute partial updates, e.g.,

$$C_{2,1} = \cancel{A_{2,1}B_{1,1}} + \cancel{A_{2,2}B_{2,1}} + \cancel{A_{2,3}B_{3,1}} + \cancel{A_{2,4}B_{4,1}}$$
$$C_{2,2} = \cancel{A_{2,1}B_{1,2}} + \cancel{A_{2,2}B_{2,2}} + \cancel{A_{2,3}B_{3,2}} + \cancel{A_{2,4}B_{4,2}}$$
$$C_{2,3} = \cancel{A_{2,1}B_{1,3}} + \cancel{A_{2,2}B_{2,3}} + \cancel{A_{2,3}B_{3,3}} + \cancel{A_{2,4}B_{4,3}}$$
$$C_{2,4} = \cancel{A_{2,1}B_{1,4}} + \cancel{A_{2,2}B_{2,4}} + \cancel{A_{2,3}B_{3,4}} + \cancel{A_{2,4}B_{4,4}}$$

- ~~Send B-row to next processor~~
  (But we'll assume this to happen)



# Performance model (same slide as for mat-vec-mul)

- **Parameters** of **parallel algorithm**:
  - "**p**": Number of processors
  - "**n**": Matrix size
  - "**r**" = n/P: Size of each matrix partition where "r" should be an integer number
- **Parameters** given by the **hardware**:
  - "**L**": Latency of network communication
  - "**B**": Bandwidth of network communication
  - "**w**": The computation time for one unit of work (mul or add operation)
- Communication:
  - Send and receive of same message takes same time
  - **Non-blocking communication** at the beginning of each iteration


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Performance model

- For **one iteration**:
  - Time for **computation**

$$T_{\text{comp}} = \underbrace{r \cdot n}_{\substack{\text{Output}\\ \text{elements}}} \cdot \underbrace{r}_{\substack{\text{Operations}\\ \text{per element}}} \cdot \underbrace{2 \cdot w}_{\text{mul + add}}$$

  - Non-blocking communication of **chunks of "x"**

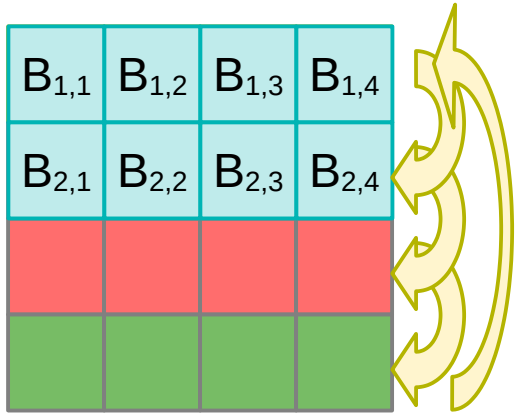
$$T_{\text{comm}} = L + \underbrace{r \cdot n}_{\substack{\text{Row of}\\ \text{matrix}}} \frac{1}{B}$$

  - The **total wallclock** time for one iteration is then

$$T_{\text{iter}} = \max\left(T_{\text{comp}}, T_{\text{comm}}\right)$$

- Time for **p iterations** (full matrix-vector multiplication:

$$\begin{aligned} T_{\text{MM}}(p) &= p \cdot \max\left(T_{\text{comp}}, T_{\text{comm}}\right) \\ &= p \cdot \max\left(r^2 \cdot n \cdot 2 \cdot w, L + \frac{r \cdot n}{B}\right) \\ &= \max\left(\frac{2n^3 w}{p}, pL + \frac{n^2}{B}\right) \end{aligned}$$




$r = \frac{n}{p}$

# Performance model (almost same as for mat-vec-mul)

For **very large matrices**:

- Investigate the performance model for very large matrices:

$$\lim_{n\to\infty} T_{MV}(n,p) = \lim_{n\to\infty} \max\left(\frac{2n^3w}{p}, pL + \frac{n^2}{B}\right)$$
$$= \lim_{n\to\infty} \frac{2n^3w}{p}$$

- Workload “perfectly” distributed

**=> Linear scaling**

For **large number of processors**:

- Investigate performance model for large number of processors:

$$\lim_{p\to\infty} T_{MV}(n,p) = \lim_{p\to\infty} \max\left(\frac{2n^3w}{p}, pL + \frac{n^2}{B}\right)$$
$$= \lim_{p\to\infty} pL + \frac{n^2}{B}$$

- The latency will increase linearly with the number of processors

**=> Sub-optimal scaling dominated by latency**


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Outline

- Warmup
- **Examples**
  - Dot product
  - Matrix-vector multiplication
  - Matrix-matrix multiplication
  - **Matrix-matrix with matrix-vector**
- Scalability comparison
- Discussion

# Matrix-vector for Matrix-matrix

What if…

- We replace our matrix-matrix algorithm by “n” matrix-vector multiplications?
- Can be easily answered with our models
- We can use our models to gain a deeper understanding:

$$
\begin{aligned}
T_{\text{MM-MV}}(p, n) &= n \cdot T_{\text{MV}}(p, n) \\
&= n \cdot \max\left(\frac{2n^2 w}{p}, pL + \frac{n}{B}\right) \\
&= \max\left(\frac{2n^3 w}{p}, pnL + \frac{n^2}{B}\right)
\end{aligned}
$$

# Matrix-vector for Matrix-matrix

- Comparing this our **standard mat-mat-mul model**:

$$T_{\text{MM-MV}}(p,n) = \max\left(\frac{2n^3 w}{p}, \boxed{pnL} + \frac{n^2}{B}\right)$$

$$T_{\text{MM}}(p,n) = \max\left(\frac{2n^3 w}{p}, \boxed{pL} + \frac{n^2}{B}\right)$$

- For **increasing problem size "n"**:
  No change in performance for large "n"
- For **increasing the number of processors "p"**:
  The "new" algorithm would be **stronger dominated by latency**

Keep in mind:
**Latency (and also bandwidth) are often the main performance limitations**
=> Try to avoid this, e.g., by "bulk communication" (gathering of smaller data packages)!


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Outline

- **Examples**
  - Dot product
  - Matrix-vector multiplication
  - Matrix-matrix multiplication
  - Matrix-matrix with matrix-vector
- **Scalability comparison**
- Discussion

# Comparison of algorithms

- **Situation**:
  - You're leading the numerics group at a company
  - You're responsible for buying a new compute cluster
  - It's up to you to determine whether a compute cluster is good enough
- **Questions**: How do the previous algorithms perform
  - for **large scale (large "p")** or
  - for **large problem size (large "n")** ?
- **Problem**:
  - The cluster doesn't yet exist
  - Hence we can't do performance studies on the real cluster
- **Approach**: Investigate this with our models!

# Choosing realistic parameters

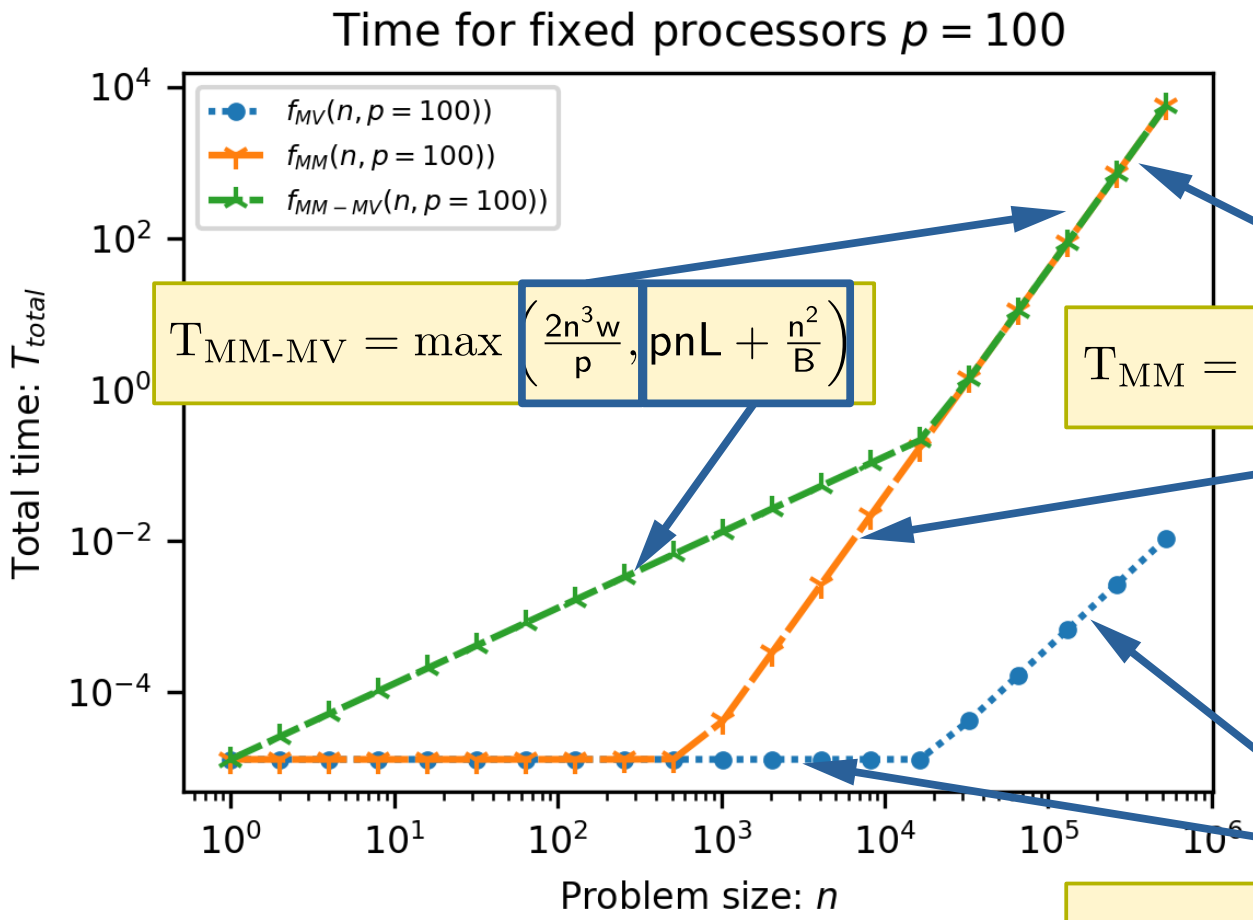
- Communication parameters of **network**:
  - (From: “Introducing 200G HDR InfiniBand Solutions”, Whitepaper, 2019)
  - “**L**”: **Latency** of network communication: $\mathrm{L} = 130\mathrm{ns} = 130^{-9}\mathrm{s}$
  - “**B**”: **Bandwidth** of network communication:
    $\mathrm{B} = 16\mathrm{Tb/s} = 16 \cdot 10^{12}\mathrm{b/s}$
    Using **double precision** with 8 bytes per Float we get $\mathrm{B} = 2 \cdot 10^{12}\mathrm{F/s}$

- Parameters for **computational part**:
  - “**w**”: The computation time for one unit of work
  - Assuming **2 GHz**, **16 cores**
  - **Fused Multiply-Add**, **8 vector elements**
  - (No superscalarity)

$$\mathrm{w} = \frac{1}{\underbrace{2 \cdot 10^9}_{\text{Ghz}} \cdot \underbrace{16}_{\text{cores}} \cdot \underbrace{2}_{\text{FMADD}} \cdot \underbrace{8}_{\text{AVX512}}}$$


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Scalability study with varying problem size "n"



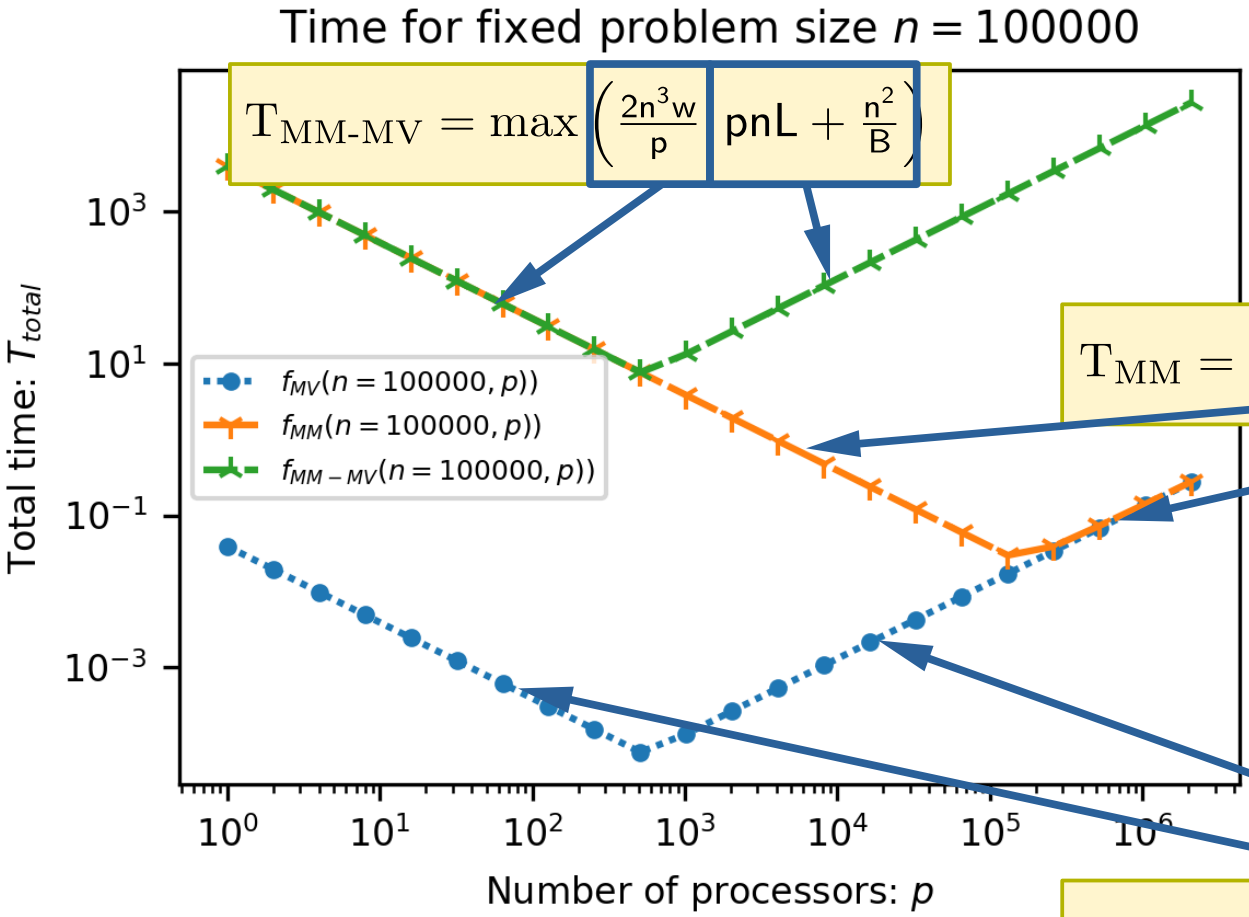

$$T_{\text{MM-MV}} = \max\left(\frac{2n^3 w}{p}, pnL + \frac{n^2}{B}\right)$$

$$T_{\text{MM}} = \max\left(\frac{2n^3 w}{p}, pL + \frac{n^2}{B}\right)$$

$$T_{\text{MV}} = \max\left(\frac{2n^2 w}{p}, pL + \frac{n}{B}\right)$$

Compute-bound for large-scale problems

# Scalability study with varying problem size "n"




$$T_{MM\text{-}MV} = \max\left(\frac{2n^3 w}{p}, pnL + \frac{n^2}{B}\right)$$

$$T_{MM} = \max\left(\frac{2n^3 w}{p}, pL + \frac{n^2}{B}\right)$$

$$T_{MV} = \max\left(\frac{2n^2 w}{p}, pL + \frac{n}{B}\right)$$

Latency-bound for large number of processors

# Outline

- Warmup
- **Examples**
  - Dot product
  - Matrix-vector multiplication
  - Matrix-matrix multiplication
  - Matrix-matrix with matrix-vector
- Scalability comparison
- **Discussion**

# Discussion

Are these models realistic?
Which simplification might render them useless?

- **Only considered ring-based network**
  => Using different network, potential congestion by other communication
- **Only considered single running application**
  => Network could be congested by communication of other applications
- **Strongly simplified work model**
  => Better with roofline model & computational density
  (Including this would already provide a very accurate model)
- and many more issues…

=> Understand the models and their limitations


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# High-Performance Computing

Based on slides of Thomas Ropars + the book "Parallel Algorithms" (by Casanova, Robert, and Legrand)

## Performance modeling of distributed memory algorithms (Part II)

**Martin Schreiber**
Université Grenobles Alpes

**part_3_81_EXTRA_perf_models_distr_mem_part_2.odp**

Université Grenoble Alpes

Inria

LABORATOIRE JEAN KUNTZMANN
MATHÉMATIQUES APPLIQUÉES - INFORMATIQUE

# Outline

- **Warmup**
  - **Example: Image processing**
  - Example: Finite differences
  - Example: Iterative solvers
- Parallel stencil algorithms
  - Greedy algorithm
  - Block-wise parallelization
  - “Row-cyclic”: Row-cyclic parallelization
  - “Bulk-cyclic”: Row-cyclic with bulk communication
  - “Block-cyclic”: Block-cyclic with bulk communication
- Discussion

# Image processing (1/2)

- **Basic image**
  - 2D array
  - Each array element stores
    - either a **grayscale** value or
    - the **red, green and blue** contributions
  - Example: 2D image

- **Stencil-based filters**
  - **Stencil** is also a 2D array
  - Examples:
    - Vertical edge detection:

| | | |
|---|---|---|
| 1 | 0 | -1 |
| 2 | 0 | -2 |
| 1 | 0 | -1 |

    - Gaussian smoother:

$\frac{1}{16}$

| | | |
|---|---|---|
| 1 | 2 | 1 |
| 2 | 4 | 2 |
| 1 | 2 | 1 |

    - Applied as a convolution:

$$y[n] = x[n] * s[n] = \sum_k x[k] \cdot s[n-k]$$


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Image processing: Example

- **Original image:**

- **Gaussian smoother filter** with a an extend of “1” would be

$$\frac{1}{16}\begin{bmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{bmatrix}$$

- **After applying the filter multiple times:**

Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Outline

- **Warmup**
  - Example: Image processing
  - **Example: Finite differences**
  - Example: Iterative solvers
- Parallel stencil algorithms
  - Greedy algorithm
  - Block-wise parallelization
  - “Row-cyclic”: Row-cyclic parallelization
  - “Bulk-cyclic”: Row-cyclic with bulk communication
  - “Block-cyclic”: Block-cyclic with bulk communication
- Discussion

# Solvers for finite differences

- Partial differential equations
  - e.g., to describe physical phenomena
  - can be solved using finite differences (on regular grids such as arrays)
- E.g. for 1st order differential operators
  - Forward differences:

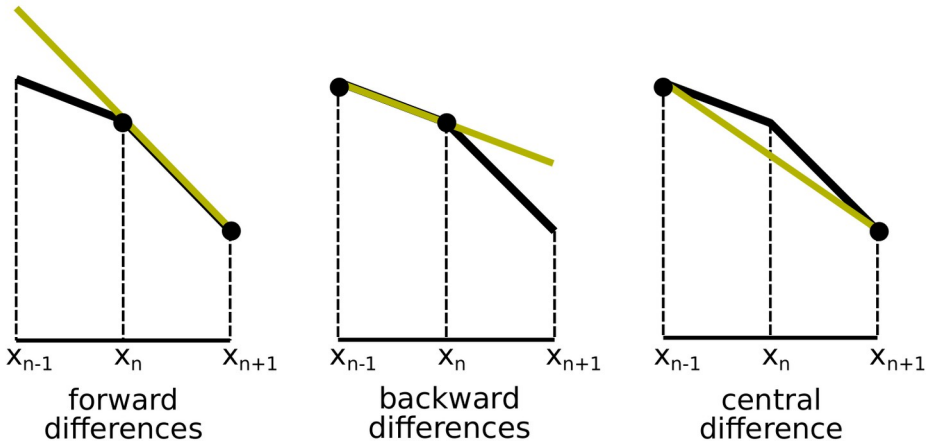
$$\frac{\partial}{\partial x} u(\mathbf{x}_n) = \frac{\mathbf{u}_{n+1} - \mathbf{u}_n}{\Delta x} + \mathcal{O}(\Delta x)$$

  - Backward differences:

$$\frac{\partial}{\partial x} u(\mathbf{x}_n) = \frac{\mathbf{u}_n - \mathbf{u}_{n-1}}{\Delta x} + \mathcal{O}(\Delta x)$$

  - Centered differences:

$$\frac{\partial}{\partial x} u(\mathbf{x}_n) = \frac{\mathbf{u}_{n+1} - \mathbf{u}_{n-1}}{2\Delta x} + \mathcal{O}(\Delta x^2)$$




Can be applied as stencils in 1D, 2D and 3D

# Outline

- **Warmup**
  - Example: Image processing
  - Example: Finite differences
  - **Example: Iterative solvers**
- Parallel stencil algorithms
  - Greedy algorithm
  - Block-wise parallelization
  - “Row-cyclic”: Row-cyclic parallelization
  - “Bulk-cyclic”: Row-cyclic with bulk communication
  - “Block-cyclic”: Block-cyclic with bulk communication
- Discussion

# Iterative solvers

- Given a linear system of equations $\mathbf{A}\mathbf{x} = \mathbf{b}$ which needs to be solved
- Assumed properties of “A”:
  - “A” is sparse (e.g., generated by finite differences)
  - “A” very large (e.g., applied to a large-scale problem)
- Solver strategy:
  - We can’t apply a Gaussian elimination
  - We can’t apply a LU decomposition
  - We can’t do anything which generates a dense matrix

  - but…
    we can try out iterative methods (if “A” has certain properties)


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Example 1: Gauss-Seidel iterative solver

- Given a (**diagonal-dominant**) matrix “A” which can be **decomposed** into

$$A = L + U$$

- “L” is the **lower triangle matrix** including the diagonal
- “U” is the **upper triangle matrix** excluding the diagonal

$$A = \underbrace{\begin{bmatrix} a_{11} & 0 & \cdots & 0 \\ a_{21} & a_{22} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ a_{n1} & a_{n2} & \cdots & a_{nn} \end{bmatrix}}_{L} + \underbrace{\begin{bmatrix} 0 & a_{12} & \cdots & a_{1n} \\ 0 & 0 & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & 0 \end{bmatrix}}_{U}$$

- Using **Gauss-Seidel** we can iteratively solve it by applying

$$\mathbf{x}^{(k+1)} = L^{-1}\left(\mathbf{b} - U\mathbf{x}^{(k)}\right)$$

… which looks indeed rather complicated

- Or alternatively:
  - For the k-th iteration:
    - **Solve row “i” of the linear system of equations** and update “$x_i$” in the vector “x”
    - Use this “x” for the next row “i+1”


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Stencil performance model

- **Algorithm**:
  - Given a 2D domain
  - Resolution of “n” in each dimension
  - Total values: $n \times n$
- **Parallelization**:
  - Distributed memory
  - Using “p” processors (distributed memory)
- We assume a **simplified stencil operation** than before

- **Special stencil access pattern**: “c” depends on applied stencils on cells “N”orth and “W”est

$$c_{new} = \text{stencil}(c_{old}, N_{new}, W_{new})$$

| | | | | |
|---|---|---|---|---|
| 1 | 2 | 3 | 4 | 5 |
| 2 | 3 | 4 | 5 | 6 |
| 3 | 4 | **N** | 6 | 7 |
| 4 | **W** | **c** | **E** | 8 |
| 5 | 6 | **S** | 8 | 9 |

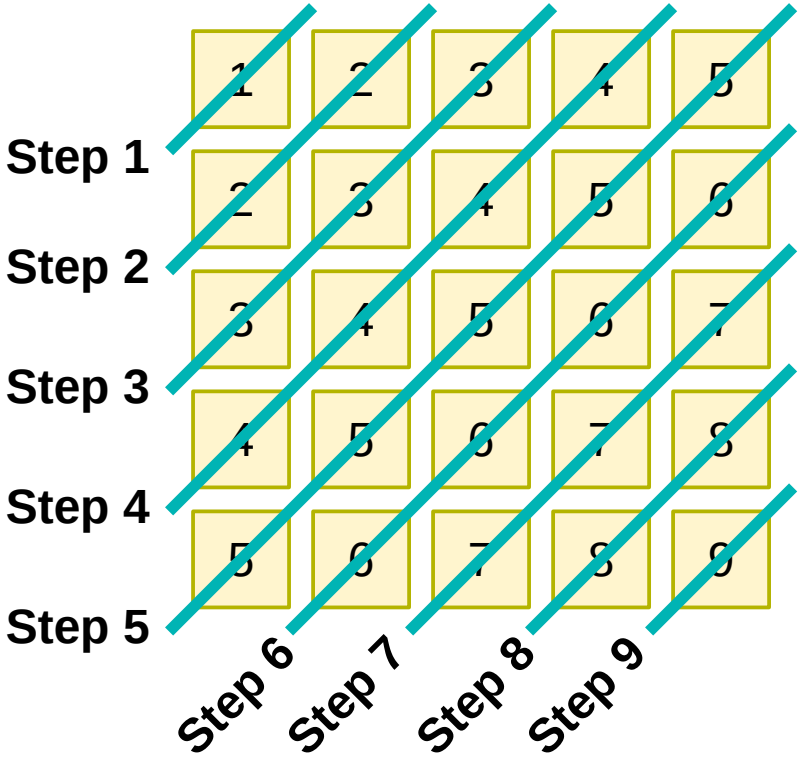

Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Outline

- Warmup
  - Example: Image processing
  - Example: Finite differences
  - Example: Iterative solvers
- **Stencil performance model**
  - **Greedy algorithm**
  - Block-wise parallelization
  - “Row-cyclic”: Row-cyclic parallelization
  - “Bulk-cyclic”: Row-cyclic with bulk communication
  - “Block-cyclic”: Block-cyclic with bulk communication
- Discussion

# Greedy algorithm

- Start at left top corner
- As soon as result is ready, send it to next processor
- On **step 5**, we use **all "p" processors** on the anti-diagonal
- **Cases:**
  - $p = n$ **Optimal** if we have **at least processors**
  - $p > n$ Leads to **idling processors**
  - $p < n$ ? => Next Sl



# Outline

- Warmup
  - Example: Image processing
  - Example: Finite differences
  - Example: Iterative solvers
- **Parallel stencil algorithms**
  - Greedy algorithm
  - **Block-wise parallelization**
  - “Row-cyclic”: Row-cyclic parallelization
  - “Bulk-cyclic”: Row-cyclic with bulk communication
  - “Block-cyclic”: Block-cyclic with bulk communication
- Discussion

# Block-wise parallelization

- **Version 1a:**
  - **Blocks of consecutive rows are assigned** to processors
  - First column processed first

| | | | | | |
|---|---|---|---|---|---|
| $P_0$ | 1 | 3 | 5 | 7 | 9 |
| | 2 | 4 | 6 | 8 | 10 |
| $P_1$ | 3 | 5 | 7 | 9 | 11 |
| | 4 | 5 | 8 | 10 | 12 |
| $P_2$ | 5 | 6 | 9 | 11 | 13 |

**Execution order:**

- $P_0$: Compute 1st columns
- $P_1$ starts as soon as $P_0$ finished 1st column, $P_0$ continues
- $P_2$: Starts as soon as P1 finishes 1st column
- etc.


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Block-wise parallelization

- **Version 1a:**
  - **Blocks of consecutive rows are assigned** to processors
  - First column processed first

| | | | | | |
|---|---|---|---|---|---|
| $P_0$ | 1 | 3 | 5 | 7 | 9 |
| | 2 | 4 | 6 | 8 | 10 |
| $P_1$ | 3 | 5 | 7 | 9 | 11 |
| | 4 | 5 | 8 | 10 | 12 |
| $P_2$ | 5 | 6 | 9 | 11 | 13 |

**Performance model:**

- $P_0$ starts with 1st column
- $P_1$ waits for $\lceil \frac{p}{n} \rceil$ steps (Number or rows of $P_1$)
- $P_2$ waits for $2 \times \lceil \frac{p}{n} \rceil$ steps
- $P_2$ has only one row to process

**Observations:**

- Single row is not processed first
- Processors do not start as early as possible


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Block-wise parallelization

- **Version 1b:**
  - **Blocks of consecutive rows are assigned** to processors
  - First column processed first

| | | | | | |
|---|---|---|---|---|---|
| $P_0$ | 1 | 2 | 3 | 4 | 5 |
| $P_1$ | 2 | 4 | 6 | 8 | 10 |
| | 3 | 5 | 7 | 9 | 11 |
| $P_2$ | 4 | 5 | 8 | 10 | 12 |
| | 5 | 6 | 9 | 11 | 13 |

**Performance model:**

- $P_0$ starts with 1st column (has only one row to process)
- $P_1$ waits for $\lfloor \frac{p}{n} \rfloor$ steps (Number or rows of $P_1$)
- $P_2$ waits for $\lceil \frac{p}{n} \rceil + \lfloor \frac{p}{n} \rfloor$ steps
- $P_2$ has only one row to process

**Observations:**

- Still **same total runtime** as before
- Again: Processors do not start as early as possible


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Outline

- Warmup
  - Example: Image processing
  - Example: Finite differences
  - Example: Iterative solvers
- **Parallel stencil algorithms**
  - Greedy algorithm
  - Block-wise parallelization
  - **“Row-cyclic”: Row-cyclic parallelization**
  - “Bulk-cyclic”: Row-cyclic with bulk communication
  - “Block-cyclic”: Block-cyclic with bulk communication
- Discussion

# Row cyclic parallelization

- **Version 2a:**
  - **Cyclic assignment of rows** to processors
  - First, **fully process each row before going to next one**

| | | | | | | |
|---|---|---|---|---|---|---|
| $P_0$ | 1 | 2 | 3 | 4 | 5 | 6 |
| $P_1$ | 2 | 3 | 4 | 5 | 6 | 7 |
| $P_2$ | 3 | 4 | 5 | 6 | 7 | 8 |
| $P_0$ | 7 | 8 | 9 | 10 | 11 | 12 |
| $P_1$ | 8 | 9 | 10 | 11 | 12 | 13 |
| $P_2$ | 9 | 10 | 11 | 12 | 13 | 14 |

## Execution order

- $P_0$: Compute 1st row
- $P_1$: Start as soon as $P_0$ finished 1st element on 1st row
- $P_2$: start as soon as $P_1$ finished processing its 1st element
- ...
- p-th row can only start if $P_0$ finished its first row!
- etc.


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Row cyclic parallelization

- **Version 2a:**
  - **Cyclic assignment of rows** to processors
  - First, **fully process each row before going to next one**

| | | | | | | |
|---|---|---|---|---|---|---|
| $P_0$ | 1 | 2 | 3 | 4 | 5 | 6 |
| $P_1$ | 2 | 3 | 4 | 5 | 6 | 7 |
| $P_2$ | 3 | 4 | 5 | 6 | 7 | 8 |
| $P_0$ | 7 | 8 | 9 | 10 | 11 | 12 |
| $P_1$ | 8 | 9 | 10 | 11 | 12 | 13 |
| $P_2$ | 9 | 10 | 11 | 12 | 13 | 14 |

**Performance:**

- For first “p” rows:
  - $P_1$: needs to wait “1” step
  - $P_2$: needs to wait “2” steps, ...
- For rows directly after them:
  - $P_0$: needs to waits “7” step
  - $P_1$: needs to waits “8” steps
  - $P_2$: needs to waits “9” steps

**Observation:**

- Total number of steps is “14”


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Row cyclic parallelization

- **Version 2b:**
  - **Cyclic assignment of rows** to processors
  - Process **columns first**

| | | | | | | |
|---|---|---|---|---|---|---|
| $P_0$ | 1 | 3 | 5 | 7 | 9 | 11 |
| $P_1$ | 2 | 4 | 6 | 8 | 10 | 12 |
| $P_2$ | 3 | 5 | 7 | 9 | 11 | 13 |
| $P_0$ | 2 | 4 | 6 | 8 | 10 | 12 |
| $P_1$ | 3 | 5 | 7 | 9 | 11 | 13 |
| $P_2$ | 4 | 6 | 8 | 10 | 12 | 14 |

**Execution order:**

- $P_0$: $1^{st}$ element of $1^{st}$ row
- $P_1$: Start as soon as $P_0$ finished $1^{st}$ element
- $P_2$: Start as soon as $P_1$ finished $1^{st}$ element
- $P_0$: Continue with element of $1^{st}$ column

# Row cyclic parallelization

- **Version 2b:**
  - **Cyclic assignment of rows** to processors
  - Process **columns first**

| | | | | | | |
|---|---|---|---|---|---|---|
| $P_0$ | 1 | 3 | 5 | 7 | 9 | 11 |
| $P_1$ | 2 | 4 | 6 | 8 | 10 | 12 |
| $P_2$ | 3 | 5 | 7 | 9 | 11 | 13 |
| $P_0$ | 2 | 4 | 6 | 8 | 10 | 12 |
| $P_1$ | 3 | 5 | 7 | 9 | 11 | 13 |
| $P_2$ | 4 | 6 | 8 | 10 | 12 | 14 |

**Performance:**

- For first “p” rows:
  - $P_1$: needs to wait “1” step
  - $P_2$: needs to wait “2” steps, ...
- For rows after it:
  - $P_0$: needs to waits “1” step
  - $P_1$: needs to waits “2” steps
  - $P_2$: needs to waits “3” steps

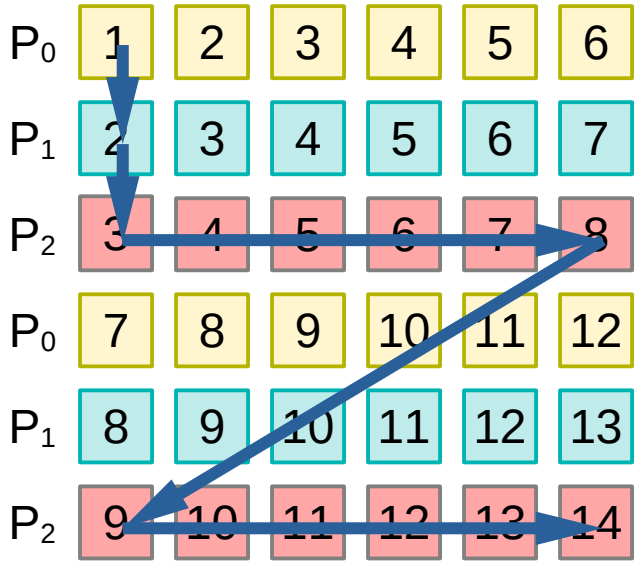
**Observation:**

- Same total number of steps as before (14)

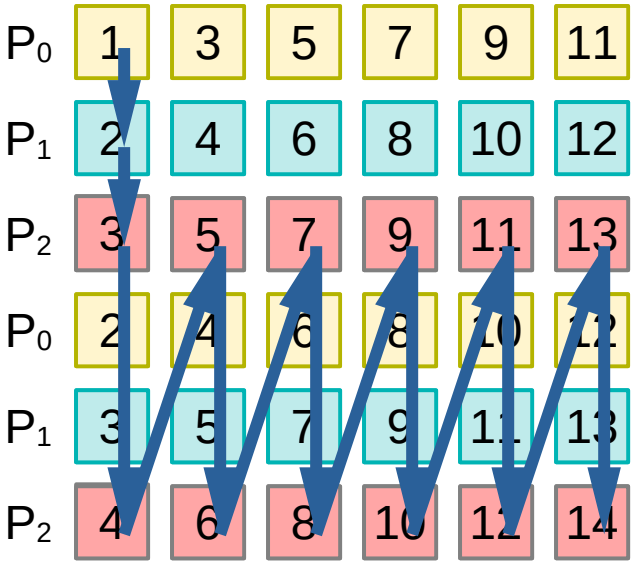

Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Critical path:
# Shortest path between first and last processed element

- Version 2a




- Version 2b




With this model, the number of steps are the same for both algorithms


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Performance model (1/2)

- Performance model:
  - “B”: Bandwidth
  - “L”: Latency
  - “$T_{cellComm}$”: Time to transfer one cell

$$T_{\mathrm{cellComm}} = L + \frac{\mathrm{sizeof(cell)}}{B}$$

  - “$T_{cellStencil}$”: Cost of executing a stencil at a cell
- Further assumption:
  - Time for receive is included in the time to send

- For each cell update:
  - **Receive** cell from row above “r-1” (left value is already available)
  - **Apply stencil**
  - **Send** cell for next row “r+1”

- Time for each cell update:

$$\begin{aligned} T_{\mathrm{cell}} &= T_{\mathrm{cellComm}} + T_{\mathrm{cellStencil}} \\ &= L + \frac{\mathrm{sizeof(cell)}}{B} + T_{\mathrm{cellStencil}} \end{aligned}$$


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Performance model (2/2)

## Time until last processor finishes:

- Time until processor starts computations:

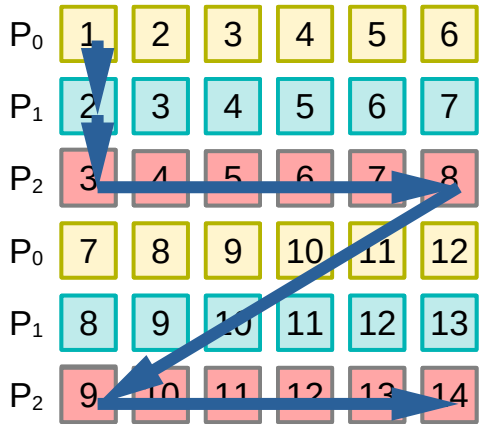
$$T_{\text{start}} = (p-1) \cdot T_{\text{cell}}$$

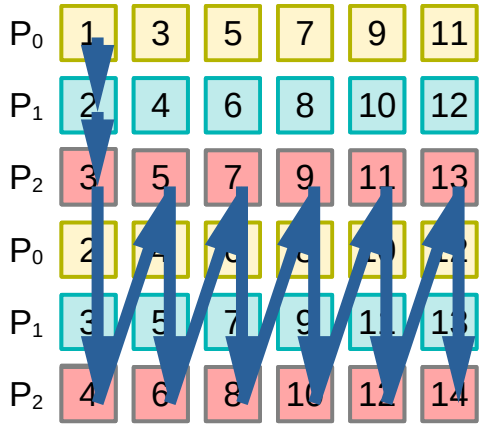
- Time for all cells computed on last processor:

$$T_{\text{last}} = \underbrace{n}_{\text{cols}} \cdot \underbrace{\frac{n}{p}}_{\text{rows}} \cdot T_{\text{cell}}$$

- In total we get

$$\begin{aligned} T_{\text{cellTotal}} &= T_{\text{start}} + T_{\text{last}} \\ &= \left((p-1) + \frac{n^2}{p}\right)\left(L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}\right) \end{aligned}$$



# Speedup (1/2)

- Time for sequential computation without any parallelization:

$$T_{\text{seq}}(n) = n^2 \cdot T_{\text{cellStencil}}$$

- For $n \gg p$ we can use the approximation

$$T^*_{\text{bulkTotal}} \approx \left(\frac{n^2}{p}\right)\left(\frac{1}{k}L + \left(\frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}\right)\right)$$

- Speedup:

$$S_{\text{cell}}(n,p) = \frac{T_{\text{seq}}(n)}{T^*_{\text{cellTotal}}(n,p)} = \frac{n^2 \cdot T_{\text{stencil}}}{\frac{n^2}{p}\left(L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}\right)}$$

$$= \frac{T_{\text{cellStencil}}}{L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}} p$$


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Speedup (2/2)

Should be 1 for linear scaling

Latency in denominator is largest term
=> Dominated by latency!!!

$$S_{\text{cell}}(n, p) = \boxed{\frac{T_{\text{cellStencil}}}{L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}}} \; p$$

**<u>Possible optimization #1</u>: Avoid frequent communication** to reduce latency impact

=> Investigate bulk communication

**<u>Possible optimization #2:</u> Reduce volume of data** to be communicated

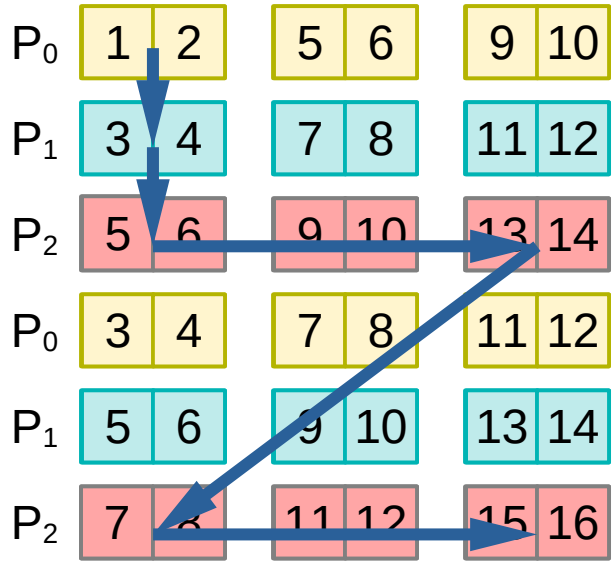
# Outline

- Warmup
  - Example: Image processing
  - Example: Finite differences
  - Example: Iterative solvers
- **Parallel stencil algorithms**
  - Greedy algorithm
  - Block-wise parallelization
  - “Row-cyclic”: Row-cyclic parallelization
  - **“Bulk-cyclic”: Row-cyclic with bulk communication**
  - “Block-cyclic”: Block-cyclic with bulk communication
- Discussion

# Bulk communication

- Idea:
  - Try to **reduce overall latency** impact by sending **less messages**, but ...
  - … resulting in **larger messages**
- This should reduce the impact of the latency term
- Model:
  - **Each bulk consists out of “k” cells**
  - => Bulk-wise computation
  - => Bulk-wise send/recv

- Variant 3a: Example for k=2 (based on variant 2b)



# Performance model (1/2)

- Performance model:
  - “B”: Bandwidth
  - “L”: Latency
  - “$T_{bulkComm}$”: Time to transfer one bulk

$$T_{\mathrm{bulkComm}} = L + \frac{\mathrm{sizeof(cell)}}{B} \cdot k$$

  - “$T_{cellStencil}$”: Cost of executing a stencil at a cell
  - “$T_{bulkStencil}$”: Stencils for a bulk

$$T_{\mathrm{bulkStencil}} = T_{\mathrm{cellStencil}} \cdot k$$

- Same assumptions as previously

- For each **bulk update**:
  - **Receive bulk** from row above “r-1” (left value is already available)
  - **Apply stencil for full bulk**
  - **Send bulk** for next row “r+1”

- Time for each **bulk update**

$$\begin{aligned} T_{\mathrm{bulk}} &= T_{\mathrm{bulkComm}} + T_{\mathrm{bulkStencil}} \\ &= L + \left( \frac{\mathrm{sizeof(cell)}}{B} + T_{\mathrm{cellStencil}} \right) \cdot k \end{aligned}$$


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Performance model (2/2)

- **Time until last processor finishes**
  - Time until processor starts computations:

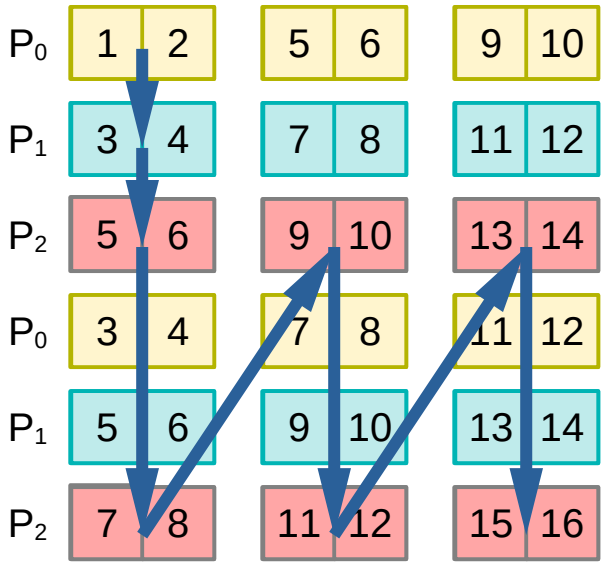
$$T_{\text{start}} = (p-1) \cdot T_{\text{bulk}}$$

  - Time for all bulks computed on last processor:

$$T_{\text{last}} = \underbrace{\frac{n}{b}}_{\text{col bulks}} \cdot \underbrace{\frac{n}{p}}_{\text{rows}} \cdot T_{\text{bulk}} = \frac{n^2}{b \cdot p} \cdot T_{\text{bulk}}$$

  - In total we get

$$\begin{aligned} T_{\text{bulkTotal}} &= T_{\text{start}} + T_{\text{last}} \\ &= \left((p-1) + \frac{n^2}{k \cdot p}\right)\left(L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}\right) \cdot k \\ &= \left(k(p-1) + \frac{n^2}{p}\right)\left(\frac{1}{k}L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}\right) \end{aligned}$$



# Comparison of parallelization approaches

## Taking the models as they are given:

- **Row-cyclic:**

$$T_{\text{cellTotal}} = \left((p-1) + \frac{n^2}{p}\right)\left(L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}\right)$$

**Additional overhead** scaling with number of processors

**Latency reduced** by the number of elements in the bulk

- **Row-cyclic bulk:**

$$T_{\text{bulkTotal}} = \left(k(p-1) + \frac{n^2}{p}\right)\left(\frac{1}{k}L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}\right)$$


Martin Schreiber, Université Grenoble Alpes / Laboratoire Jean-Kuntzmann

# Speedup (1/2)

- Time for sequential computation without any parallelization:

$$T_{\text{seq}}(n) = n^2 \cdot T_{\text{cellStencil}}$$

- For $n \gg p$ we can use the approximation

$$T^*_{\text{cellTotal}}(n,p) \approx \frac{n^2}{p}\left(L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}\right)$$

- Speedup:

$$S_{\text{bulk}}(n,p) = \frac{T_{\text{seq}}(n)}{T^*_{\text{bulkTotal}}(n,p)} = \frac{n^2 \cdot T_{\text{stencil}}}{\frac{n^2}{p}\left(\frac{1}{k}L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}\right)}$$

$$= \frac{T_{\text{cellStencil}}}{\frac{1}{k}L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}}\, p$$

# Speedup (2/2)

Should be 1 for linear scaling

Latency factor can be reduced by increasing size of bulk size

$$S_{\text{bulk}}(n,p) = \boxed{\frac{T_{\text{cellStencil}}}{\frac{1}{k}L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}}} \quad p$$

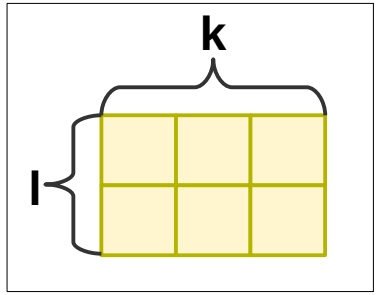
**<u>Possible optimization #2:</u>**
**Reduce volume of data**
to be communicated

# Outline

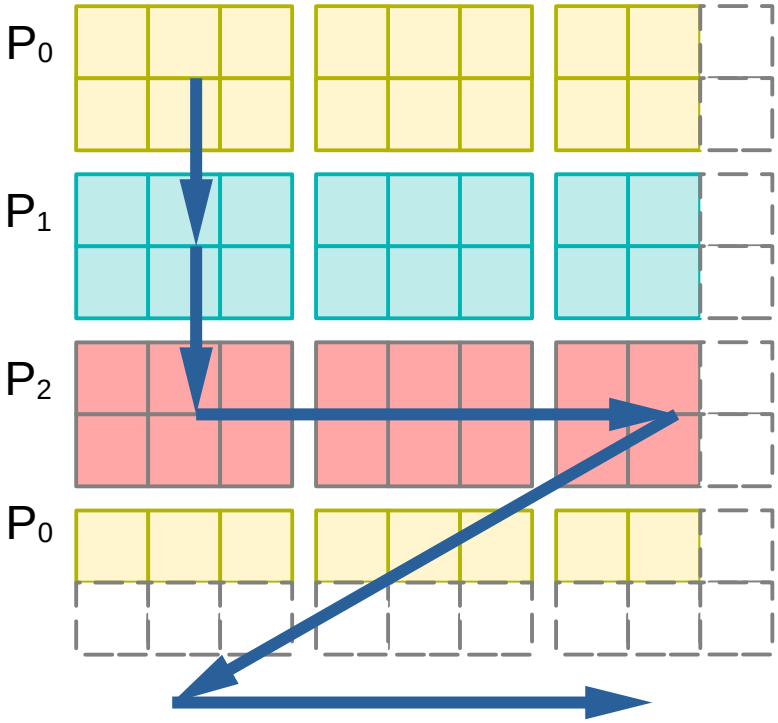
- Warmup
  - Example: Image processing
  - Example: Finite differences
  - Example: Iterative solvers
- **Parallel stencil algorithms**
  - Greedy algorithm
  - Block-wise parallelization
  - “Row-cyclic”: Row-cyclic parallelization
  - “Bulk-cyclic”: Row-cyclic with bulk communication
  - **“Block-cyclic”: Block-cyclic with bulk communication**
- Discussion

# Block-cyclic communication (with bulks)

- Idea:
  - Try to **reduce bandwidth** requirement impact by sending **less data volume**
  - This should **reduce the impact of the bandwidth-related term**
- Model: In addition to bulk
  - **Cyclic assignment of consecutive "l" rows to processors**
  - => Block-wise computation
  - => **Avoid communication**
  - => Still **bulk-wise send/recv**




- Variant 4a: Example for **k=3**, **l=2** (based on variant 2b)



# Performance model (1/2)

- Performance model:
  - "B": Bandwidth
  - "L": Latency
  - "$T_{blockComm}$" block

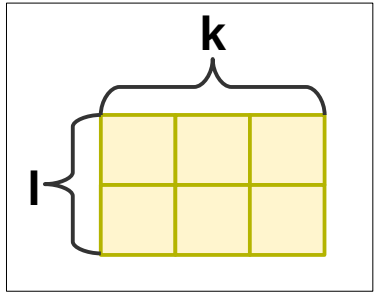
$$T_{\mathrm{blockComm}} = L + \frac{\mathrm{sizeof(cell)}}{B} \cdot k$$

(same as "$T_{bulkComm}$")

  - "$T_{cellStencil}$": Cost of executing a stencil at a cell
  - "$T_{blockStencil}$": Stencils for a block

$$T_{\mathrm{blockStencil}} = T_{\mathrm{cellStencil}} \cdot k \cdot l$$

- Same assumptions as previously




- For each **block update**:
  - **Receive <u>bulk</u>** from row above "r-1" (left value is already available)
  - **Apply stencil for full block**
  - **Send <u>bulk</u>** for next row "r+1"

- Time for each **block updat**

$$T_{\mathrm{block}} = T_{\mathrm{blockComm}} + T_{\mathrm{blockStencil}}$$
$$= L + \left(\frac{\mathrm{sizeof(cell)}}{B} + T_{\mathrm{cellStencil}} \cdot l\right) k$$

# Performance model (2/2)

- **Time until last processor finishes**
  - Time until processor starts computations:

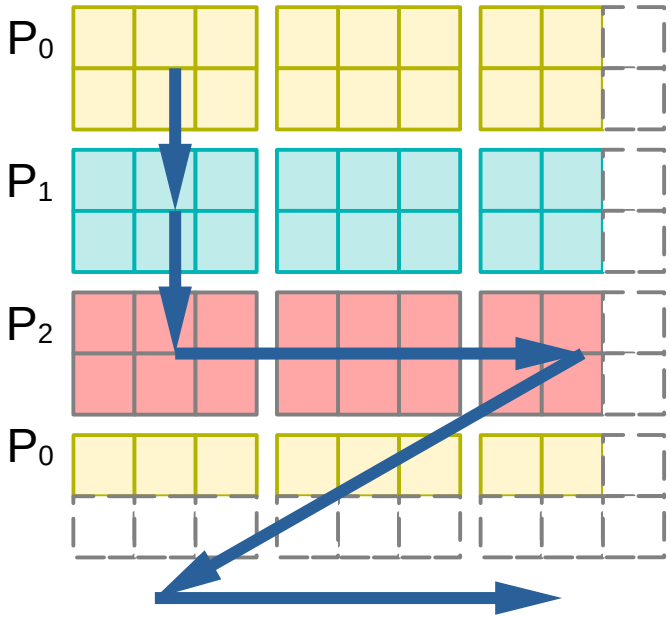
$$T_{\text{start}} = (p-1) \cdot T_{\text{block}}$$

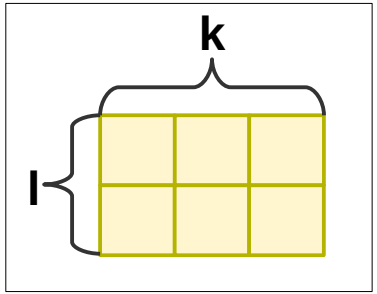
  - Time for all bulks computed on last processor:

$$T_{\text{last}} = \underbrace{\frac{n}{k}}_{\text{col blocks}} \cdot \underbrace{\frac{n}{l \cdot p}}_{\text{row blocks}} \cdot T_{\text{block}} = \frac{n^2}{k \cdot l \cdot p} \cdot T_{\text{block}}$$

  - In total we get

$$\begin{aligned} T_{\text{blockTotal}} &= T_{\text{start}} + T_{\text{last}} \\ &= \left((p-1) + \frac{n^2}{k \cdot l \cdot p}\right)\left(L + \left(\frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}} \cdot l\right) \cdot k\right) \\ &= \left(k \cdot l \cdot (p-1) + \frac{n^2}{p}\right)\left(\frac{1}{k \cdot l} L + \left(\frac{\text{sizeof(cell)}}{B \cdot l} + T_{\text{cellStencil}}\right)\right) \end{aligned}$$



# Comparison of parallelization approaches

## Taking the models as they are given:

- **“Bulk”** (Row-cyclic bulk)

$$T_{\text{bulkTotal}} = \left(k(p-1) + \frac{n^2}{p}\right)\left(\frac{1}{k}L + \frac{\text{sizeof(cell)}}{B} + T_{\text{cellStencil}}\right)$$

**Additional overhead** scaling slightly with “l”

**Latency further reduced** by the number of rows in block

**Bandwidth reduced** by the number of rows in block

- **“Block”** (Row-cyclic blocks)

$$T_{\text{blockTotal}} = \left(k \cdot l \cdot (p-1) + \frac{n^2}{p}\right)\left(\frac{1}{k \cdot l}L + \left(\frac{\text{sizeof(cell)}}{B \cdot l} + T_{\text{cellStencil}}\right)\right)$$

# Summary

- Block cyclic leads to:
  - Reduction of the required bandwidth
  - In addition: reduction of total latency!
- Startup costs are high for large number of rows per block “r”
- **<u>Always</u>** optimize for latency and bandwidth reduction (at least in most of the cases)

# Outline

- Warmup
  - Example: Image processing
  - Example: Finite differences
  - Example: Iterative solvers
- Parallel stencil algorithms
  - Greedy algorithm
  - Block-wise parallelization
  - “Row-cyclic”: Row-cyclic parallelization
  - “Bulk-cyclic”: Row-cyclic with bulk communication
  - “Block-cyclic”: Block-cyclic with bulk communication
- **Discussion**

# Conclusions

- Improve latency-bound algorithms:
  - Send data in bulk
  - Sending data early to avoid having idle processors

- Improve bandwidth-bound algorithms (if possible):
  - Assigning blocks of data to processors to limit the amount of communication
  - Also leads to reduction of latency (since less bulks will be sent)

- Cyclic data distribution
  - can increase load balancing between processors and
  - reduce idle time

# Further reading

Section 4.1 and 4.3 of the book "Parallel Algorithms" (by Casanova, Robert, and Legrand).